Hadoop & Spark - ISEN 2025/2026

Chapter 4: Why limit ourselves to flat files?

Goals

In this chapter we will see:

- File format
- Definitions of different concepts about data applications & Cloud
- Data Lakes and Data Warehouse

Goals

Can you give me the file format you know?

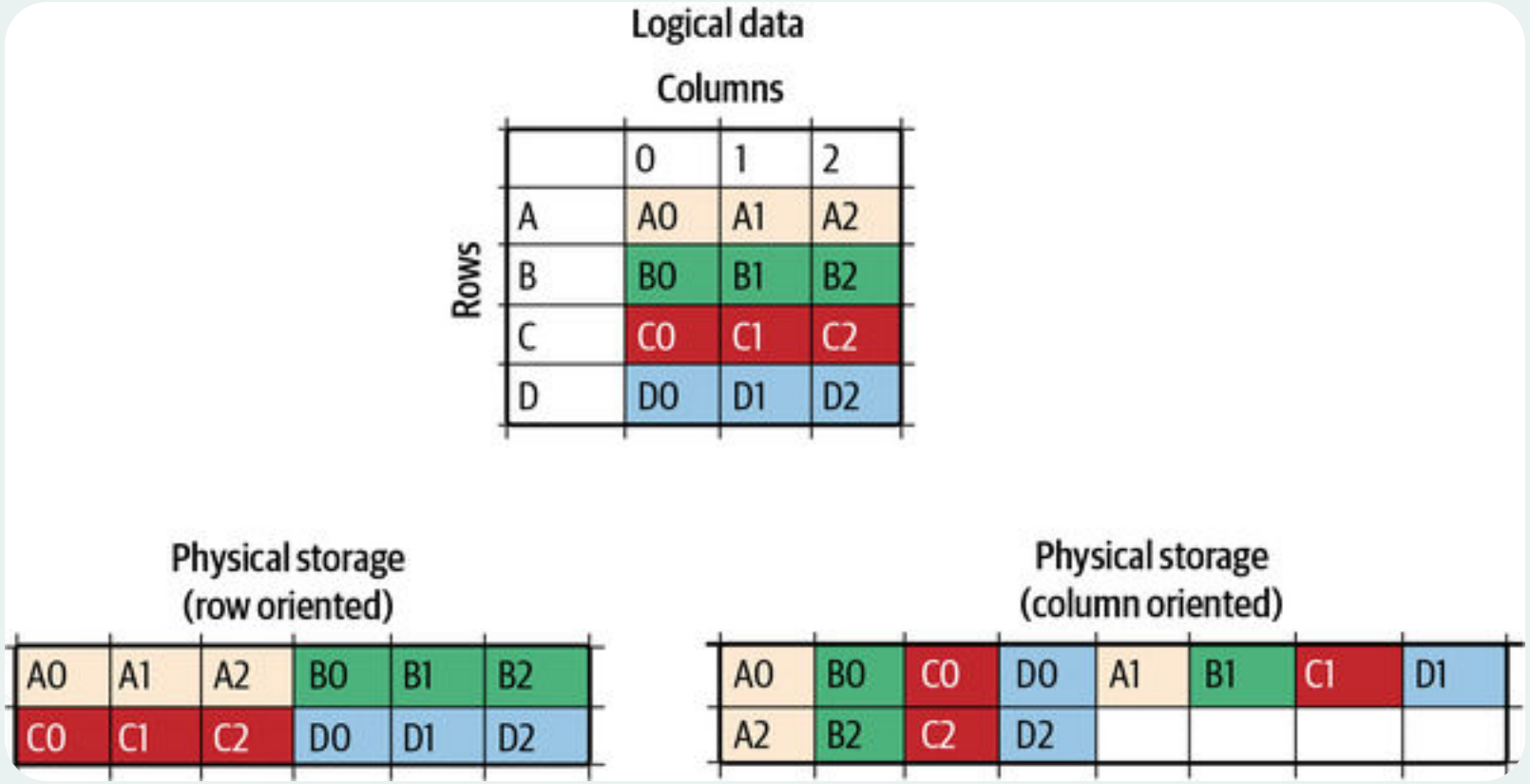


Common file format

Here is a list of the commonly used formats in data and data science:

- csv
- Parquet
- ORC
- JSON
- txt
- xlsx
- Avro
- ...

Row-Oriented vs Column-Oriented File Formats



Row-Oriented File Formats



The data is stored row by row. They include **csv**, **txt**, **Avro** & **JSON**.

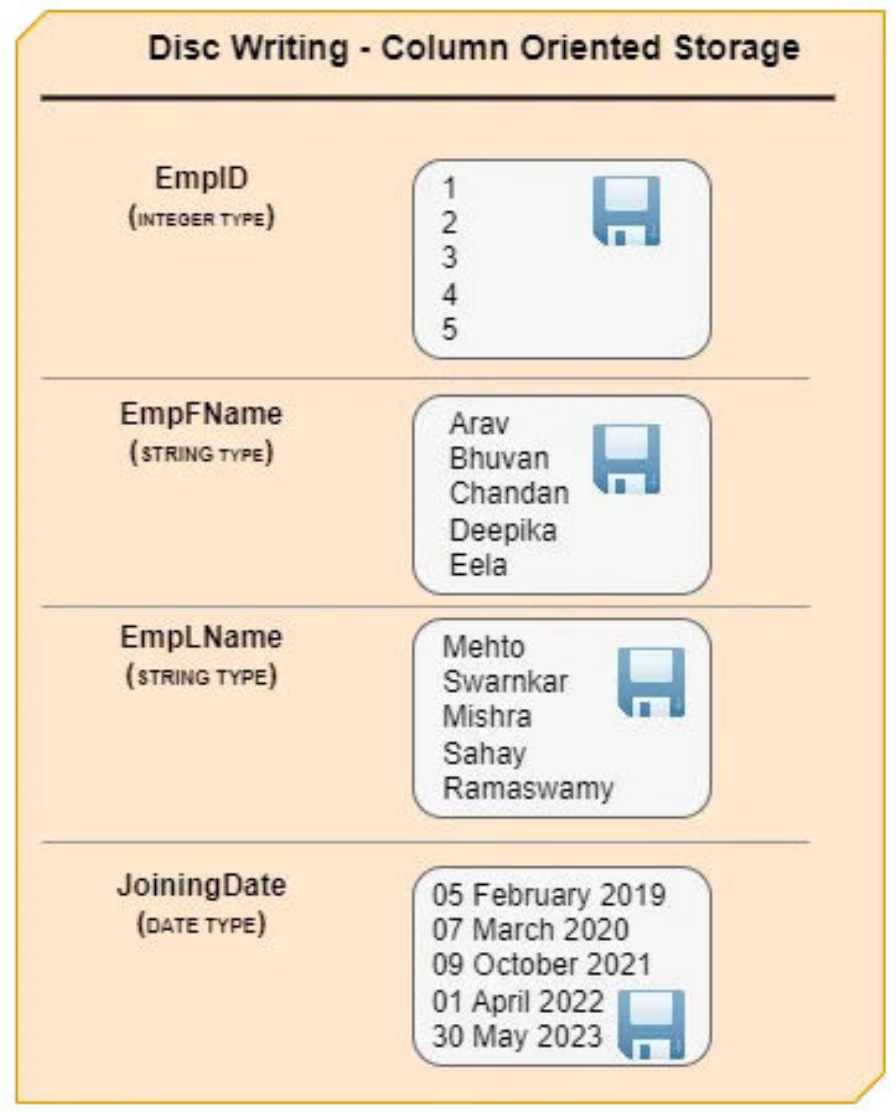
Advantages of Row-Oriented Formats ✓

- Simple and human-readable
- Fast for writing data
- Suitable for transactional processing (OLTP)

Disadvantages of Row-Oriented Formats ✗

- Less storage and memory efficient
- Limited support for complex data types
- Slower for analytical queries

Column-Oriented File Formats



The data is stored column by column they include **Parquet**, **Delta** & **ORC**

Advantages of Column-Oriented Formats ✓

- Efficient storage and compression
- Supports complex data types and schema evolution
- Fast for analytical queries (OLAP)

Disadvantages of Column-Oriented Formats ✗

- More complex file structure
- May require additional processing resources

Choosing a File Format

- Consider storage efficiency, query performance, and data types
- Evaluate compatibility with processing frameworks and tools
- Understand your data access patterns and use cases

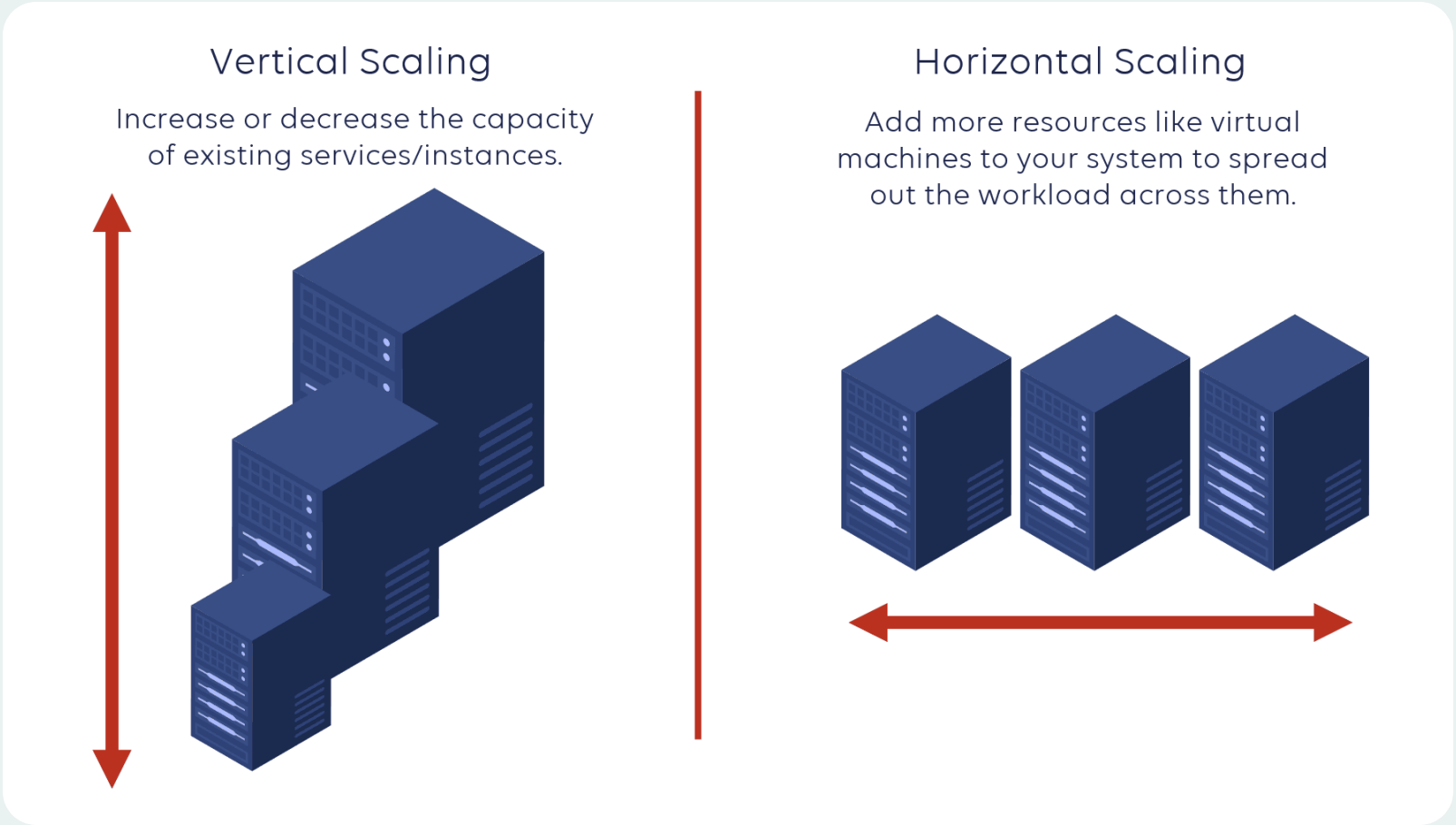
We are done with common file formats overview 🎉

Now let's get some generic definitions



Vertical & Horizontal scaling

Vertical scaling refers to increasing a system's capacity by adding more resources, such as CPU or memory, to a single node, while horizontal scaling involves adding more nodes to a system to distribute the workload across multiple machines.



Availability & redundancy

Key Concepts in Reliable Systems:

Availability

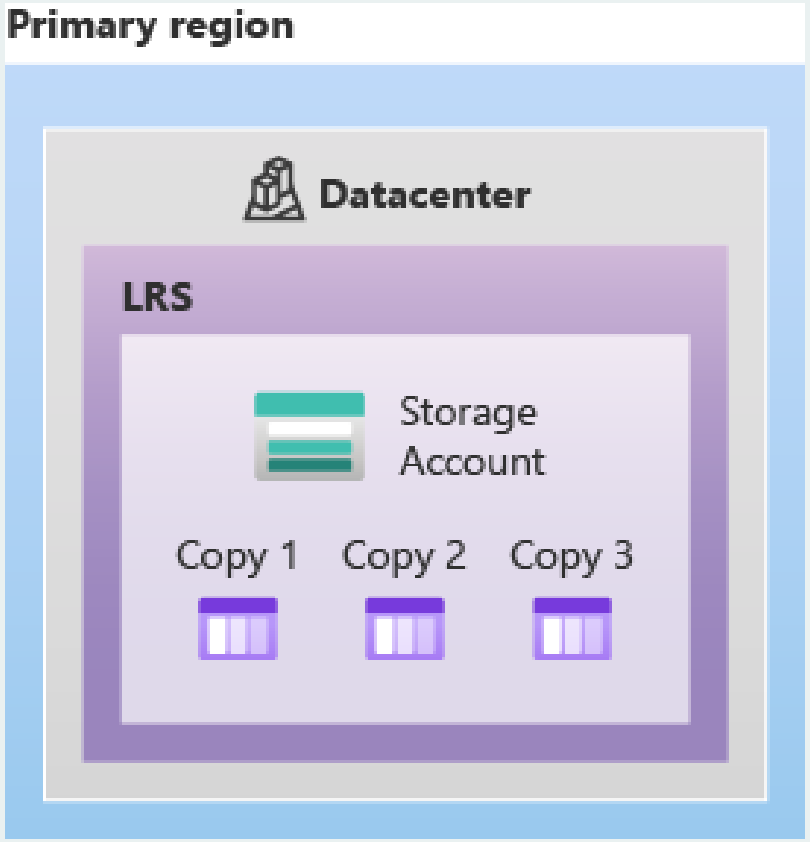
- The proportion of time a system is operational and accessible
- Typically measured as a percentage (e.g., 99.9% availability)

Redundancy

- Duplication of critical system components or resources
- Ensures continued operation in case of failures

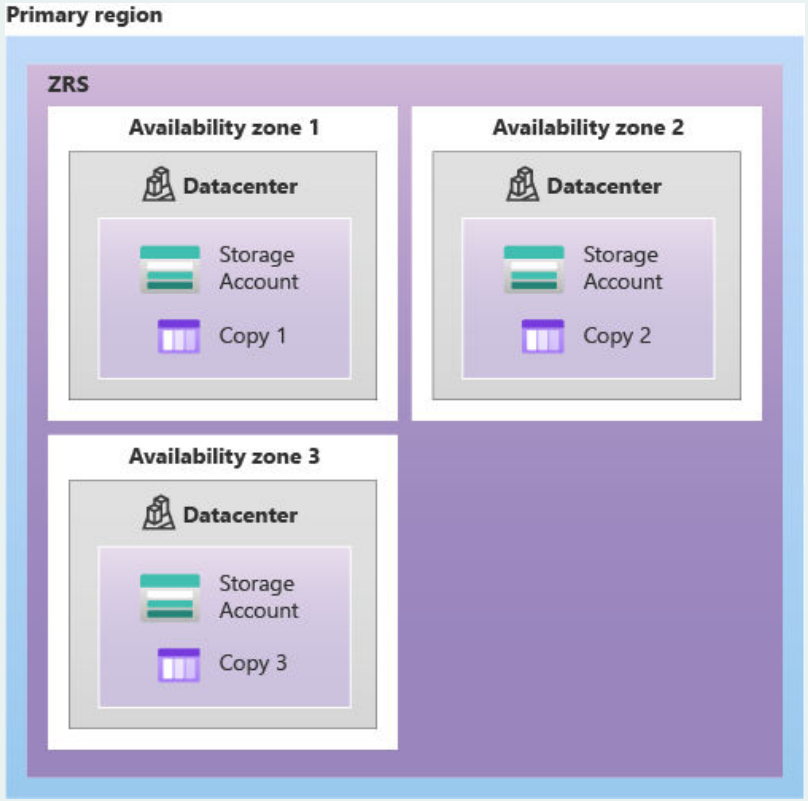
Example of redundancy in Azure Cloud Storage

1. **Locally Redundant Storage (LRS)**
    - Data replicated 3 times within a single data center
    - Protects against hardware failures



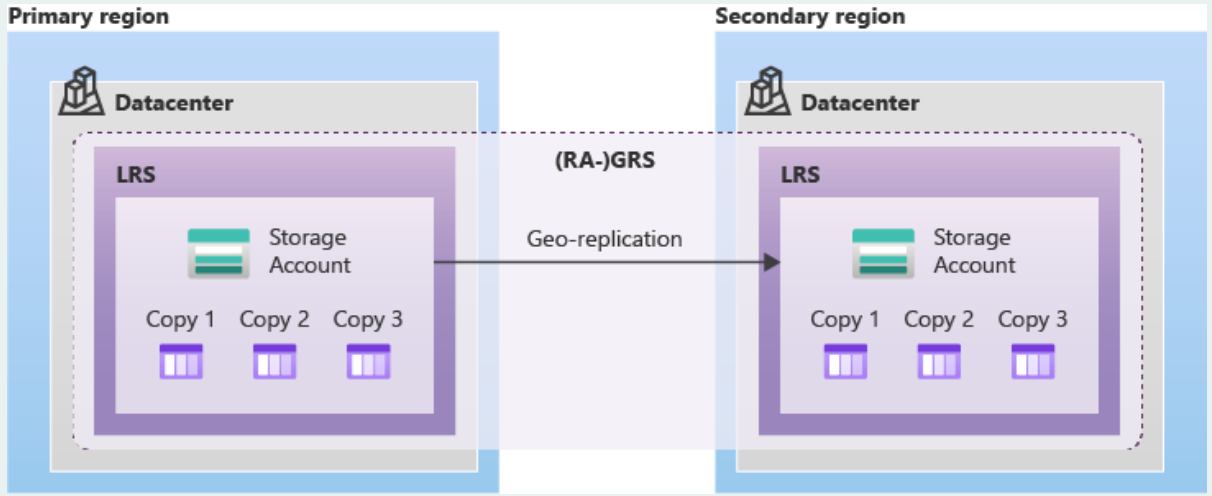
Example of redundancy in Azure Cloud Storage

2. **Zone-Redundant Storage (ZRS)**
    - Data replicated across 3 availability zones within a single region
    - Protects against data center and hardware failures



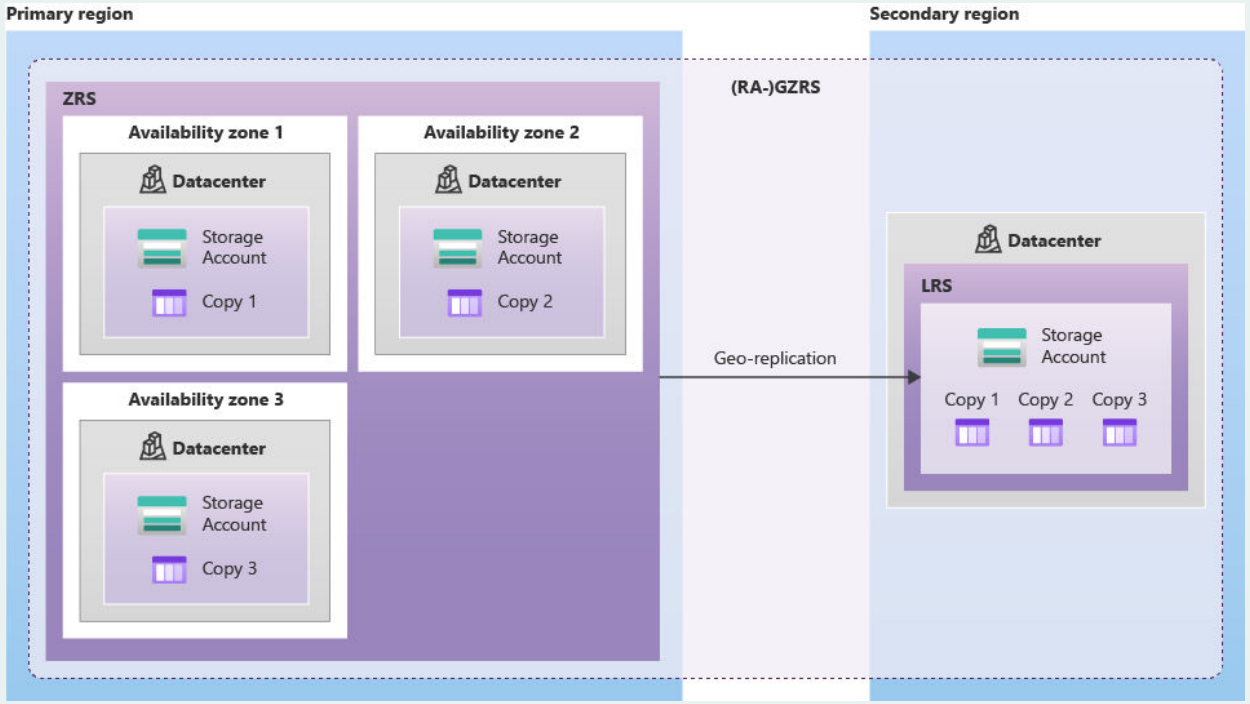
Example of redundancy in Azure Cloud Storage

3. **Geo-Redundant Storage (GRS or RA-GRS)**
    - Data replicated to a secondary region
    - Provides 6 copies of data (3 in primary and 3 in secondary region)
    - Protects against regional outages
    - For RA-GRS: same as GRS, with read access to the secondary region's replicas



Example of redundancy in Azure Cloud Storage

4. **Geo-Zone-Redundant Storage (GZRS)**
   - Data is stored across 3 availability zones in the primary region
   - Data is also replicated in a secondary region
   - Provides 6 copies of data (3 across zones in primary region, 3 in secondary region)
   - Protects against data center, hardware, and regional outages
   - For RA-GZRS: with read access to the secondary region's replicas



Continuity

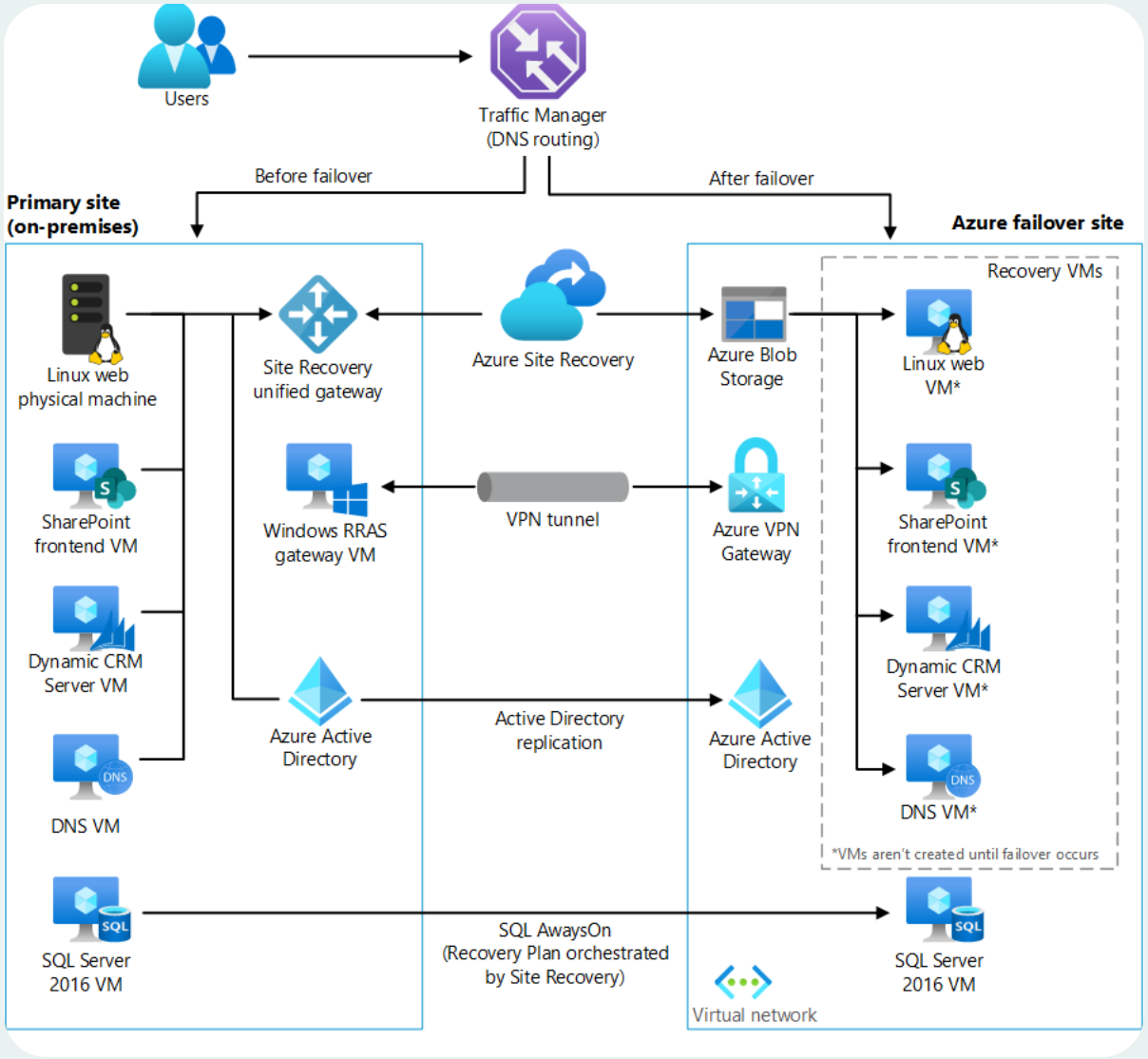
Fault Tolerance:

- The ability of a system to continue functioning despite failures or errors
- Achieved through redundancy, error detection, and recovery mechanisms
- This includes backups of both computes and data

Disaster Recovery:

- Set of policies, procedures, and tools to recover and restore IT systems and data
- Ensures business continuity in the event of natural disasters, technical failures, or human errors
- Minimizes downtime, data loss, and financial impact on the organization

Example of disaster recovery in Azure



Network latency

One aspect of the Cloud that is often overlooked at first glance is the network latency.

The time taken for data to travel from one point to another in the network affects the performance and responsiveness of cloud applications.

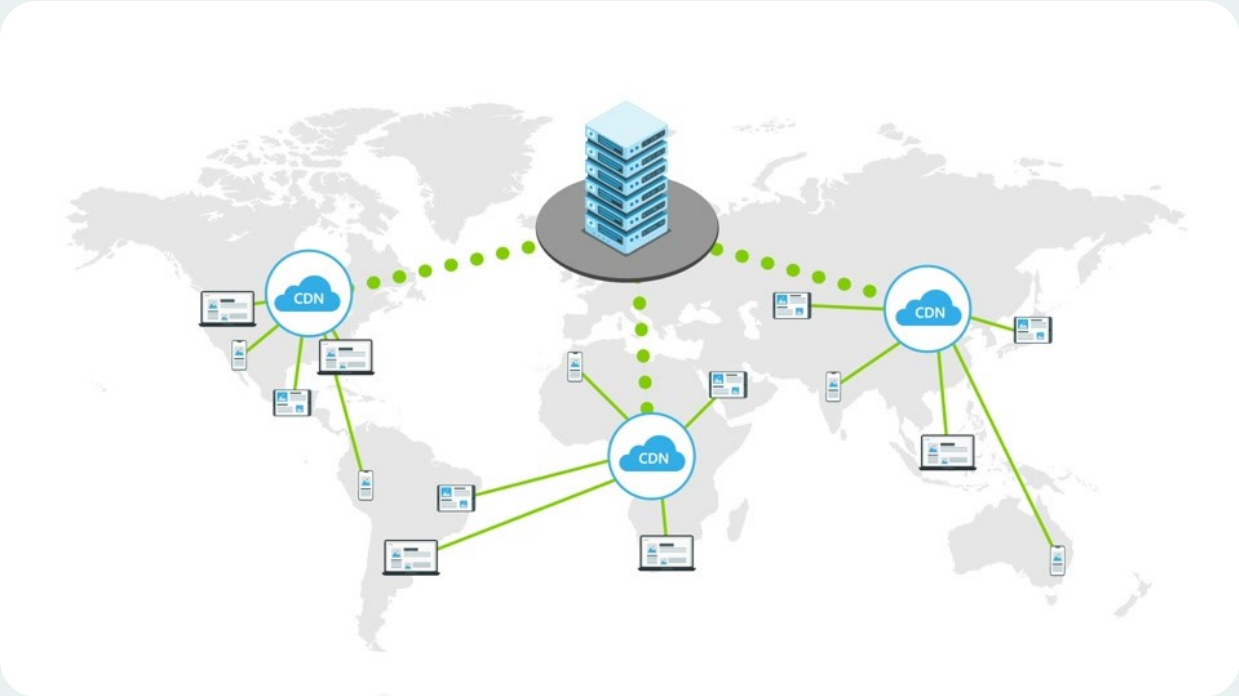
You have to consider 3 key factors:

- **Geographical distance**: Longer distances between data centers and users increase latency
- **Infrastructure quality**: Modern and well-maintained networks reduce latency
- **Network congestion**: High traffic can increase latency during peak times

Network latency

Latency issues can be addressed multiple ways, including:

- Location of the data centers closer to the end users
- Optimize application performance and network traffic (by reducing unnecessary exchanges and limiting movement of data for example)
- Use Content Delivery Networks (CDNs) to cache data in various locations



Analytical vs Transactional

You have to understand that in data, there are often two very different and distinct needs:

- **Analytical**: refers to the process of examining, cleaning, transforming, and modeling data to extract valuable insights, discover patterns and trends, and support decision-making. Analytical data is typically used for reporting, forecasting, and strategic decision-making in various industries.
- **Transactional**: refers to the real-time processing of data related to day-to-day business operations, such as sales, purchases, and customer interactions. Transactional data is typically used for managing business processes, ensuring data consistency, and maintaining data integrity.

As a result, historically we have two kinds of data processing OLAP & OLTP

OLAP (Online Analytical Processing)

OLAP is a category of software tools that provides analysis of data stored in a database. It's designed for complex queries and **multidimensional** analysis, where **the focus is on data aggregation**, pattern discovery, and trend analysis.

- **Purpose**: Support decision-making processes
- **Data**: Historical, aggregated, read-heavy
- **Operations**: Complex queries, analytical calculations
- **Examples**: Data warehouses, business intelligence systems

OLTP (Online Transactional Processing)

OLTP is a class of systems that manage transaction-oriented applications. It's designed for real-time processing of **day-to-day business tasks**, where the focus is on **data integrity, consistency, and speed**.

- **Purpose**: Support daily business operations
- **Data**: Current, detailed, write-heavy
- **Operations**: Simple, fast, atomic transactions
- **Examples**: Order management systems, banking systems

Summary of OLAP vs OLTP

Key Differences	OLAP	OLTP
Data Structure	Multidimensional data model	Relational data model
Data Volume	Large volumes of historical and aggregated data	Smaller volumes of current and detailed data
Query Types	Complex analytical queries	Simple transactional queries
Response Time	Longer due to query complexity	Fast and real-time execution
Concurrency	Optimized for read-heavy operations	Optimized for write-heavy operations
Focus	Data analysis and decision support	Day-to-day transaction processing

ACID

Transactional databases almost always follow the rules of ACID:

- Atomicity
- Consistency
- Isolation
- Durability

ACID transactions guarantee that a database will be in a consistent state after running a group of operations.



1. Atomicity

- Ensures that a transaction is completed as a unit
- Guarantees the integrity of the whole transaction
- Example: Querying customer details (name, age, location, earning potential)

2. Consistency

- Ensures that data validation rules are followed
- Only permitted data is written
- Example: Adding a new customer record (right name and age, but invalid location)

3. Isolation

- Ensures that concurrent transactions do not affect each other
- Example: Two users trying to add the same customer to the database

4. Durability

- Guarantees that data is available after a successful transaction
- Example: Customer data remains intact even after power failure or hardware outage

ACID Databases: Summary

- ACID properties ensure data integrity and reliability
- Atomicity, Consistency, Isolation, and Durability are critical for transaction processing
- Databases often rely on ACID compliance to power critical day-to-day operations and decision-making

Slowly Changing Dimensions (SCD)

SCD refers to a technique used to handle and maintain historical data when changes occur in dimension data over time.
It helps preserve data accuracy and ensures that analytical reports reflect the correct historical context at a given point in time

There are five main types of SCDs:

- Type 0: Retain Original
- Type 1: Overwrite
- Type 2: Add New Row
- Type 3: Add New Attribute
- Type 4: Add History Table

Type 0: Retain Original

- No changes to dimension data
- Historical data remains unchanged
- Not suitable for tracking changes

This one is uncommon as changes are not possible after the creation of a line.

Type 1: Overwrite

- Update dimension data with new values
- No history of previous values is maintained
- Suitable for corrections and minor updates

Type 1 Slowly Changing Dimension

Product Dim (Source)		
Product Name	**Product ID**	**Product Descr**
10 inch box	010	10 inch ~~glued~~ box 10 inch **pasted** box
12 inch box	012	12 inch glued box

Product Dim (Target)			
Product Name	**SID**	**Source Product ID**	**Product Descr**
10 inch box	0001	010	10 inch **pasted** box
12 inch box	0002	012	12 inch glued box

Type 2: Add New Row

- Create a new row for changed data
- Maintain historical data by adding versioning information
- Suitable for tracking complete history

Type 2 Slowly Changing Dimension

Product Dim (Source)		
Product Name	Product ID	Product Descr
12 inch box	012	12 inch glued box
10 inch box	010	10 inch ~~glued~~ box
		10 inch **pasted** box

Product Dim (Target)					
SID	Source Product ID	Product Name	Product Descr	EFF_ START_DT	EFF_ END_DT
0001	012	12 inch box	12 inch glued box	Jan-01-1753	Dec-31-9999
0002	010	10 inch box	10 inch glued box	Jan-01-1753	May-12-06
0003	**010**	**10 inch box**	**10 inch pasted box**	**May-12-06**	**Dec-31-9999**

Type 3: Add New Attribute

- Add a new attribute to store previous values
- Limited history, only stores current and previous value
- Suitable for tracking limited changes

Type 3 Model – Slowly Changing Dimension

Before The Change

Vendor Dimension Table

Vendor Key	Vendor Name	Old Phone	Current Phone	Effective Date
001	At-Your-Service Plumbing		202-555-8639	2/1/2001



After The Change

Vendor Dimension Table

Vendor Key	Vendor Name	Old Phone	Current Phone	Effective Date
001	At-Your-Service Plumbing	202-555-8639	858-555-6555	12/15/2008

Type 4: Add History Table

- Create a separate history table for changed data
- Maintain historical data in a separate table
- Suitable for complex scenarios with multiple historical values

SCD Type 4 current table

cust_id	customer_number	first_name	last_name
1	10001	Context	Wonderful

SCD Type 4 history table

cust_id	customer_number	first_name	last_name	effective_date
1	10001	Kontext	Wonderful	2022-01-01
1	10001	Context	Wonderful	2022-07-01

SCD combined types

Sometimes you may come across Type 5, 6, 7, and other variants. They are often combinations of other Types from 0 to 4.

For example, one of the most used of this combined type is the type 6 (= 1+2+3)

The Supplier table starts out with one record for our example supplier:

Supplier_Key	Row_Key	Supplier_Code	Supplier_Name	Current_State	Historical_State	Start_Date	End_Date	Current_Flag
123	1	ABC	Acme Supply Co	CA	CA	2000-01-01T00:00:00	9999-12-31T23:59:59	Y

The Current_State and the Historical_State are the same. The optional Current_Flag attribute indicates that this is the current or most recent record for this supplier.

When Acme Supply Company moves to Illinois, we add a new record, as in Type 2 processing, however a row key is included to ensure we have a unique key for each row:

Supplier_Key	Row_Key	Supplier_Code	Supplier_Name	Current_State	Historical_State	Start_Date	End_Date	Current_Flag	
123	1	ABC	Acme Supply Co	IL	CA	2000-01-01T00:00:00	2004-12-22T00:00:00	N	Changed (type1)
123	2	ABC	Acme Supply Co	IL	IL	2004-12-22T00:00:00	9999-12-31T23:59:59	Y	Created (type2)

We overwrite the Current_State information in the first record (Row_Key = 1) with the new information, as in Type 1 processing. We create a new record to track the changes, as in Type 2 processing. And we store the history in a second State column (Historical_State), which incorporates Type 3 processing.

Little test

What will be the new table if the company moves to New York (NY)? (Still in SCD type 6)

Supplier_Key	Row_Key	Supplier_Code	Supplier_Name	Current_State	Historical_State	Start_Date	End_Date	Current_Flag
123	1	ABC	Acme Supply Co	IL	CA	2000-01-01T00:00:00	2004-12-22T00:00:00	N
123	2	ABC	Acme Supply Co	IL	IL	2004-12-22T00:00:00	9999-12-31T23:59:59	Y

Little test: Answer

Supplier_Key	Row_Key	Supplier_Code	Supplier_Name	Current_State	Historical_State	Start_Date	End_Date	Current_Flag
123	1	ABC	Acme Supply Co	IL	CA	2000-01-01T00:00:00	2004-12-22T00:00:00	N
123	2	ABC	Acme Supply Co	IL	IL	2004-12-22T00:00:00	9999-12-31T23:59:59	Y

Supplier_Key	Row_Key	Supplier_Code	Supplier_Name	Current_State	Historical_State	Start_Date	End_Date	Current_Flag
123	1	ABC	Acme Supply Co	NY	CA	2000-01-01T00:00:00	2004-12-22T00:00:00	N
123	2	ABC	Acme Supply Co	NY	IL	2004-12-22T00:00:00	2008-02-04T00:00:00	N
123	3	ABC	Acme Supply Co	NY	NY	2008-02-04T00:00:00	9999-12-31T23:59:59	Y

Historic (type3)

Changed (type1)

Created (type2)

SCD: Summary

- Slowly Changing Dimensions are essential for maintaining historical data in data warehouses, especially for data science and Machine Learning applications.
- Five main types, each with different approaches to handling changes
- Choosing the appropriate SCD type depends on the specific requirements and desired level of historical data tracking

Wiki

Data Architectures

Now that we are done with the generic Cloud definitions, let's see the main data architectures, starting with an out-of-date model: the **on-premise Data Warehouse**.

Traditional Data Processing and Insights

Understanding Data needs in the past:

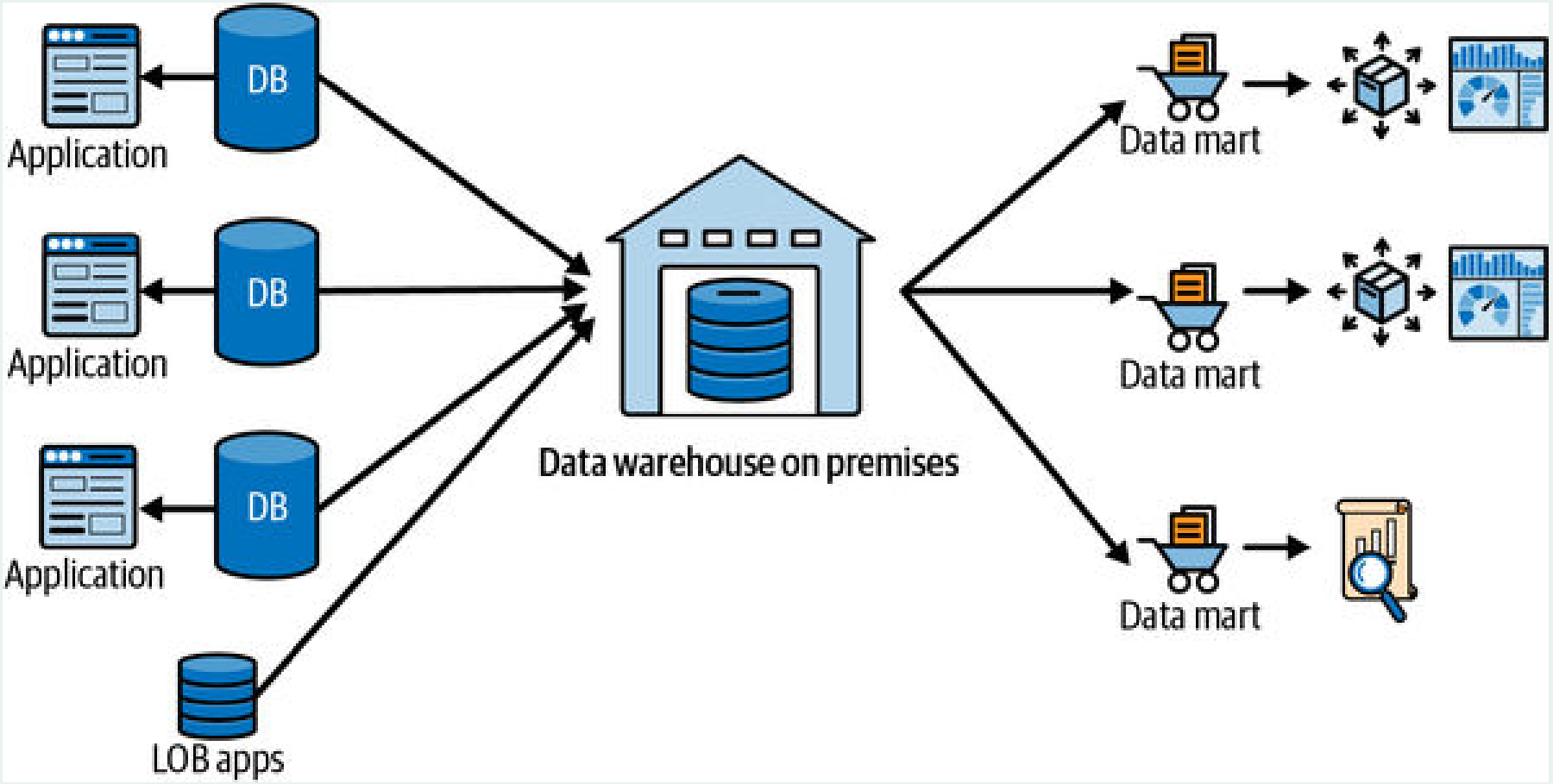
1. Business problem-centric approach
    - Identify the problem to be solved
    - Define a structure for data that can help solve the problem
    - Collect or generate the data that adheres to the structure
2. Data storage and processing
    - Store data in an OLTP database (e.g., Microsoft SQL Server)
    - Transform and store data in OLAP databases for analysis
3. Insight generation
    - Build dashboards and queries from OLAP databases to solve business problems

On-Premises Data Processing Layers (1/2)

Three Key Layers in a Traditional On-premise Data Architecture:

1. **Enterprise Data Warehouse**
    - Data storage component with database and metadata
    - Describes the data stored in the database
2. **Data Marts**
    - Segments of the enterprise data warehouse
    - Business- or topic-focused databases with transformed data
3. **Consumption/Business Intelligence (BI) Layer**
    - Visualization and query tools for BI analysts
    - Generates insights from data marts or warehouse

On-Premises Data Processing Layers (2/2)



Limitations of On-Premises Data Warehouse Solutions (1/3)

Highly Structured Data

- On-premises data warehouses expect data to be highly structured
- Not realistic due to varied data sources (IoT, social media, video/audio files) and formats (JSON, CSV, PNG, etc.)
- Strict structure cannot always be enforced

Limitations of On-Premises Data Warehouse Solutions (2/3)

Siloed Data Stores

- Multiple copies of the same data are stored in specialized data stores
- High cost of storing and copying data
- Expensive and error-prone process, resulting in inconsistent data versions across data stores

Limitations of On-Premises Data Warehouse Solutions (3/3)

Hardware Provisioning for Peak Utilization

- On-premises data warehouses require installation and maintenance of hardware
- Must plan ahead for peak utilization (e.g., fiscal year closing, holiday sales)
- Increased Total Cost of Ownership (TCO) due to underutilized hardware

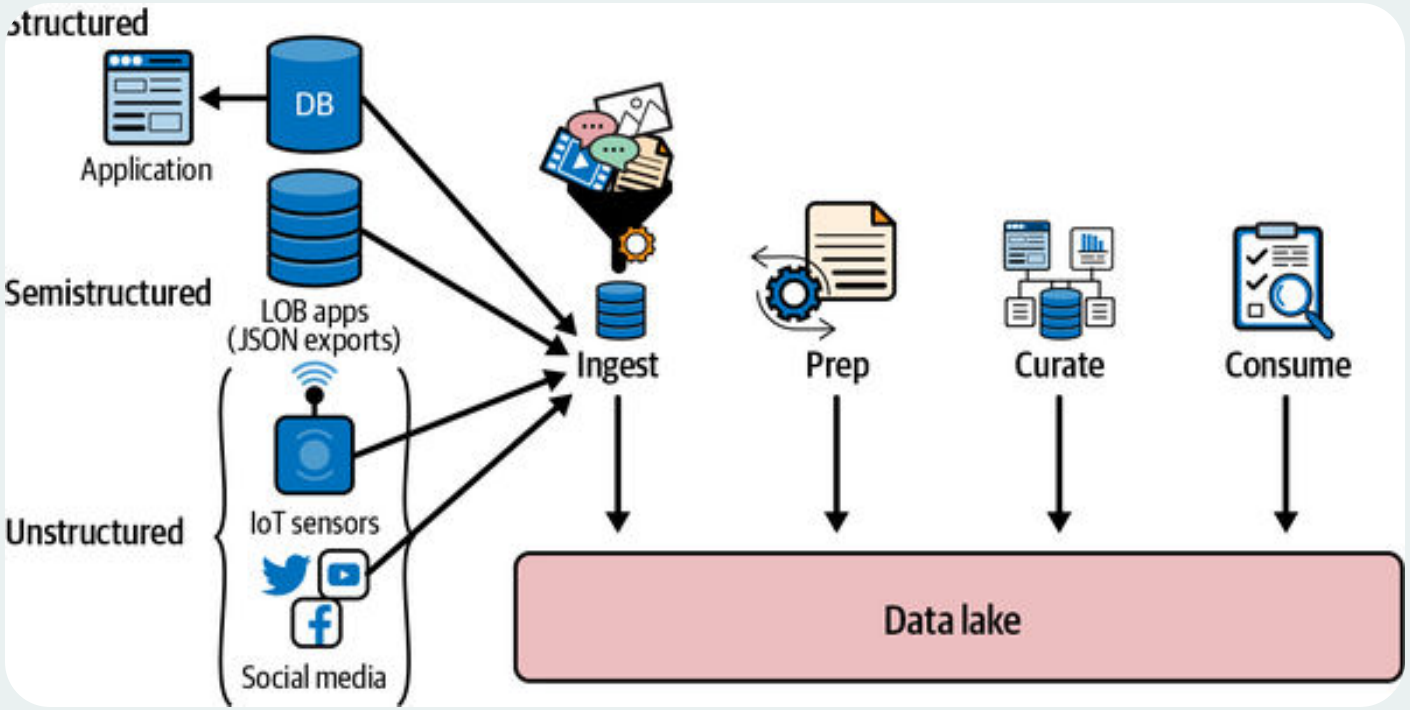
Summary

- On-premises data warehouse solutions have limitations due to highly structured data, siloed data stores, and hardware provisioning for peak utilization.
- These limitations result in increased costs, data inconsistency, and inflexibility in handling varied data sources and formats.

You probably saw it coming, but this led us to another kind of implementation:
the **Cloud Data Lake** architecture.

Cloud Data Lake

The Cloud Data Lake is designed for the explosive growth of data with no assumptions about data source, format, size, or quality in opposition to its on-premise counterpart.



Benefits of a Cloud Data Lake (1/5)

No Restrictions on Data

- Designed to ingest, store, and process all types of data (Structured, semi-structured, unstructured, relational, non-relational ...)
- No restrictions on source, size, or structure
- Low-cost storage for both real-time and batch data

Benefits of a Cloud Data Lake (2/5)

Single Storage Layer with no Silos

- Processing occurs on data in the same store
- Eliminates specialized data stores and lowers cost
- Reduces errors from moving data between storage systems

Flexibility of running diverse compute

- Decouples computing and storage
- Run various data processing tools on the same storage layer
- Supports BI queries, machine learning, data science, and domain-specific computations

Benefits of a Cloud Data Lake (4/5)

Pay for What You Use

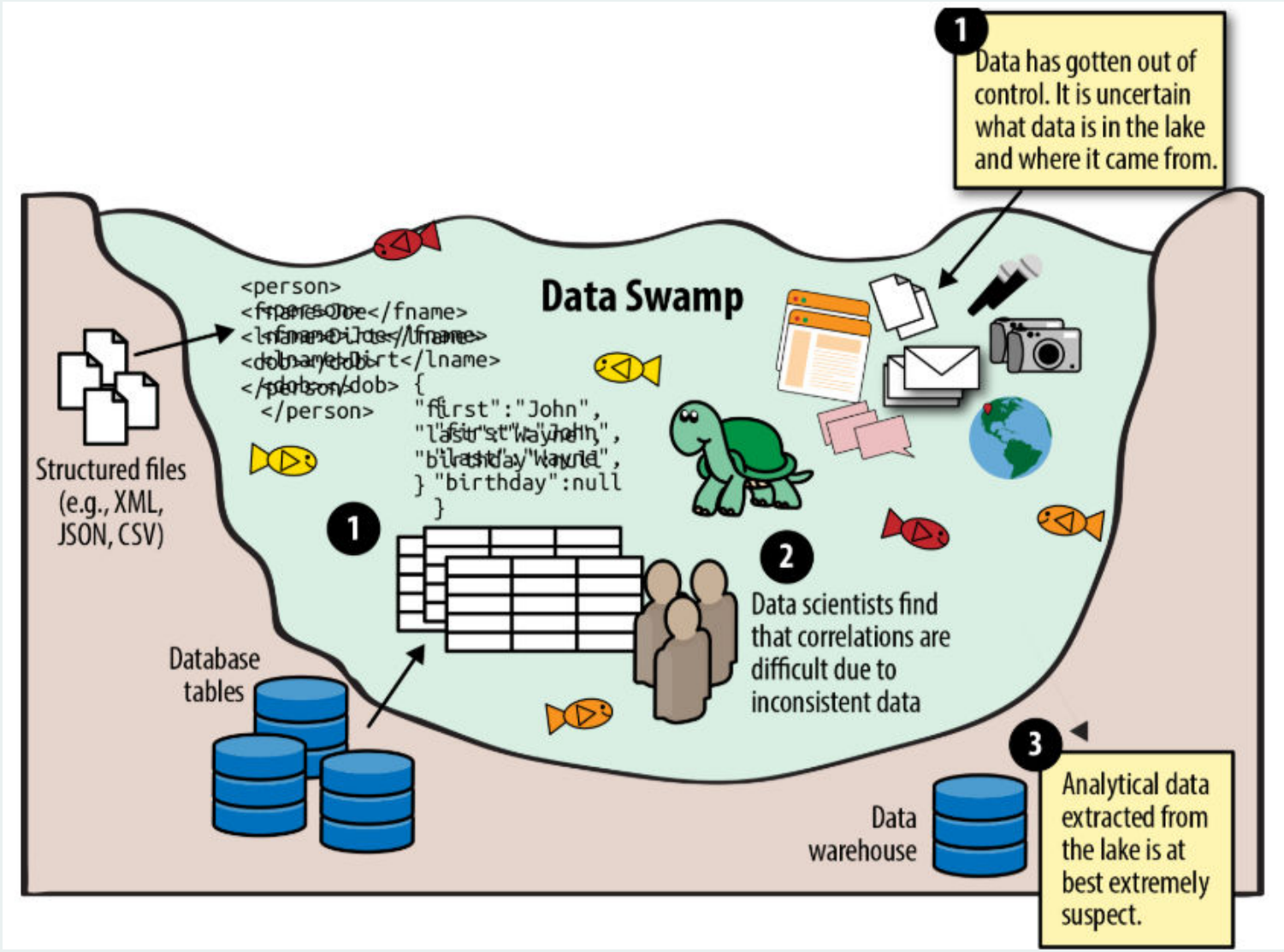
- Cloud services scale up and down elastically
- Create and delete processing systems on demand
- Reduces Total Cost of Ownership (TCO)

Independently Scale Compute and Storage

- Compute and storage are separate resources
- Scale resources depending on the need
- Affordable storage and on-demand compute resources offer economy at scale

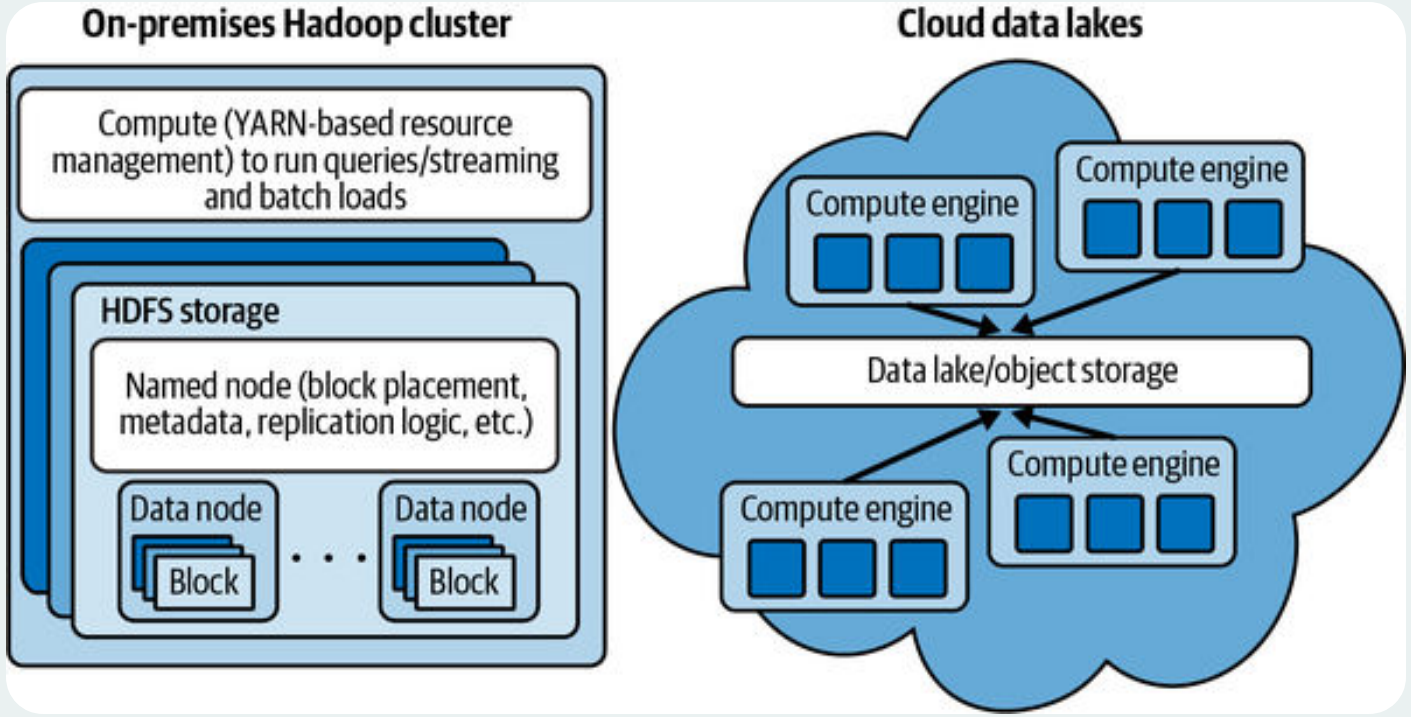
The issue with Data Lakes

This permissive structure comes with both advantages and disadvantages but **the main issue with the Datalake** is that it can become a huge and impenetrable mess if there is **no proper governance or cataloging**. We often refer to this as a 'Data Swamp'.



Summary of a Cloud Data Lake

- Cloud Data Lake Architecture offers benefits such as flexibility, scalability, cost-efficiency, and support for diverse data and computations.
- It addresses the limitations of traditional data warehouse architectures, making it an attractive solution for modern data processing needs.
- The need for governance (metadata, cataloging, control...) is high due to its inherent lack of structure.



Data Lake Architectures

Let's see an example of 3 common Data Lakes architectures:
Medallion, **Lambda** and **Kappa**

Other data lake architectures might exist, but these are some of the most common ones.

Each architecture has its own set of pros and cons, and the choice depends on the specific requirements of a project.

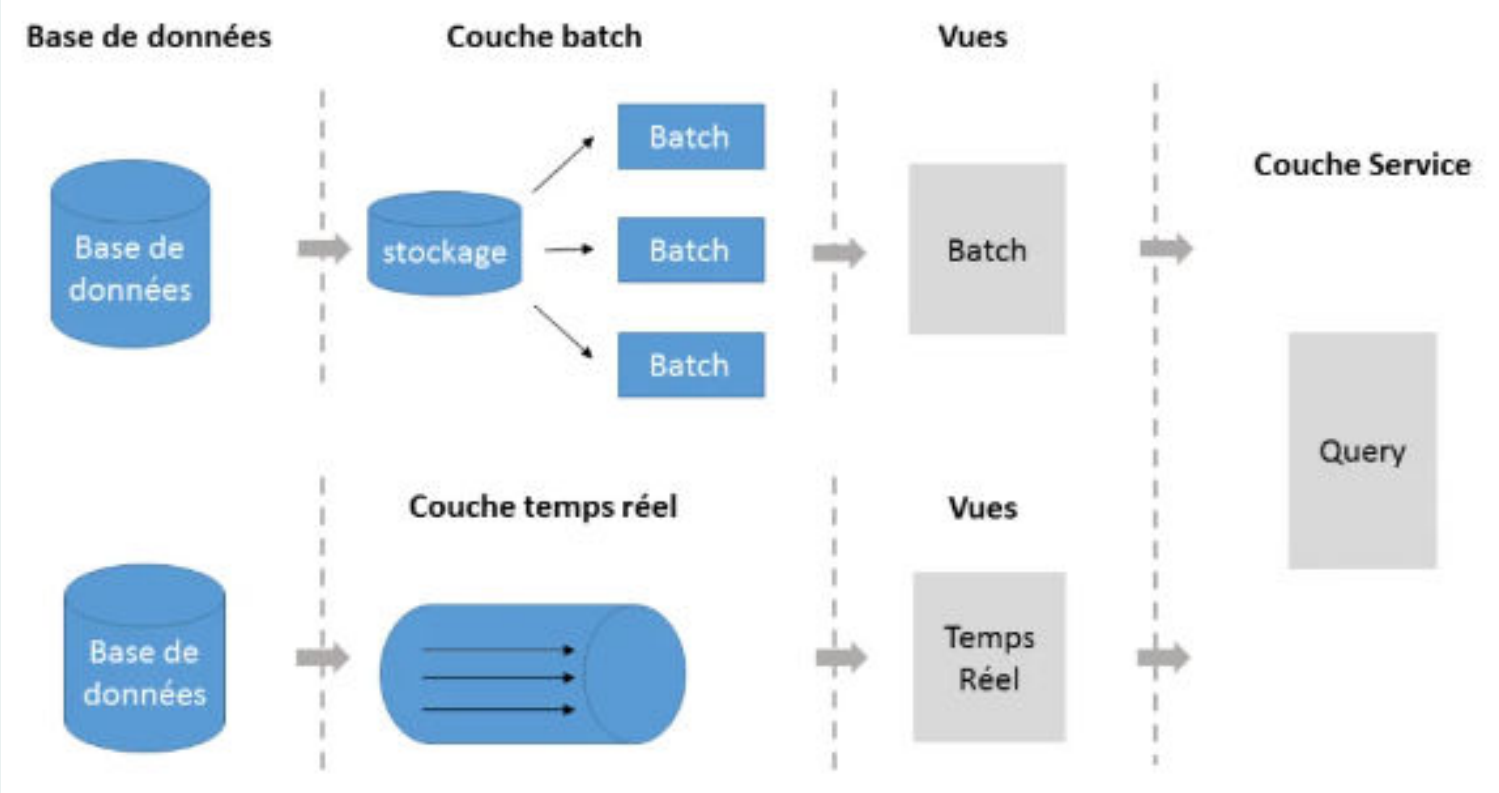
1. Medallion Architecture

- **Bronze Layer**:
  - Data is ingested in its **raw format**: No changes or transformations applied
  - Data is stored **as-is** for traceability
- **Silver Layer**:
  - Data is **cleaned**, **transformed**, **structured** and sometimes **enriched**
  - **Data quality** checks are applied
- **Gold Layer**:
  - Data is aggregated and optimized for business insights. Ready for visualization and reporting
  - High-level summaries and KPIs are created



2. Lambda Architecture

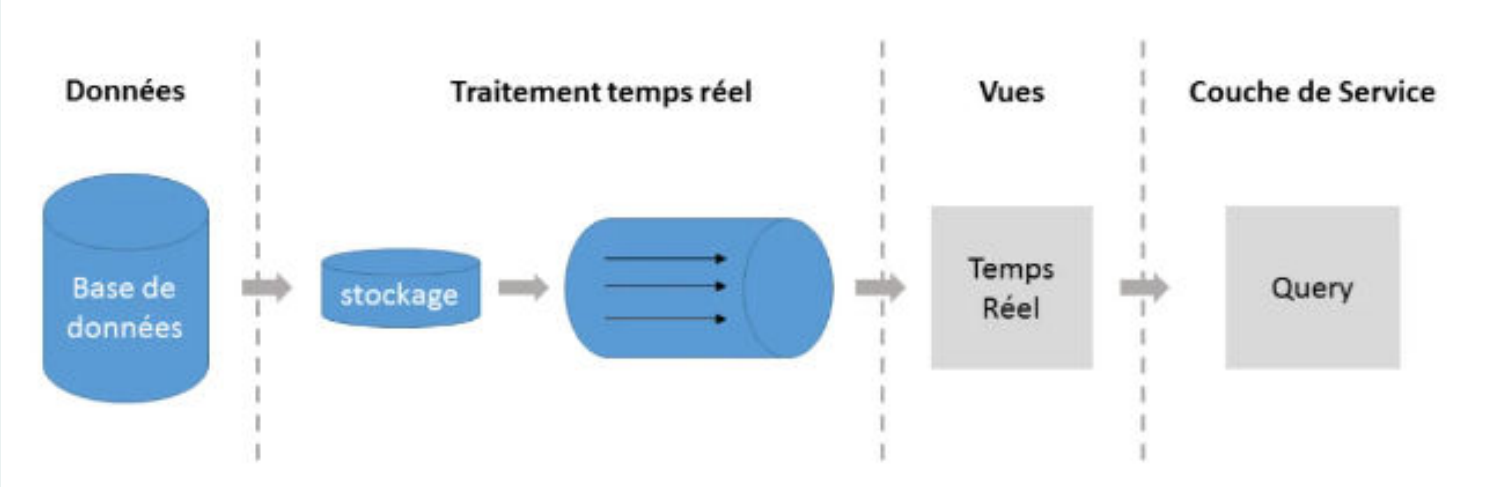
- **Batch Layer**:
  - Stores **historical** data processed in large **batches**
  - **High-latency**, **high-throughput** processing suitable for l**ong-term storage**
- **Speed Layer**:
  - Data is processed **in real-time**
  - **Low-latency**, **high-throughput** processing suitable for **time-sensitive** use cases
- **Serving Layer**:
  - Provides a **unified view** of the data
  - Allows users to query both historical and real-time data



3. Kappa Architecture

(Basically a simplified version of the Lambda Architecture)

- **Stream Layer**:
  - Data is processed in **real-time** using stream processing
  - Eliminates the need for a separate batch layer
  - Simplifies the architecture
- **Serving Layer**:
  - Provides a unified view of the data
  - Allows users to query both historical and real-time data



Another alternative: the Cloud Data Warehouse

A **Cloud Data Warehouse** is an enterprise data storage solution offered as a managed service on public clouds, designed for elastic scaling and optimized for data ingestion, analytics, and BI analytics.

The Data Warehouse is **structured** with a **defined schema** in opposition to the Datalake.

It has faster performance and lower cost of ownership than traditional on-premises data warehouses.

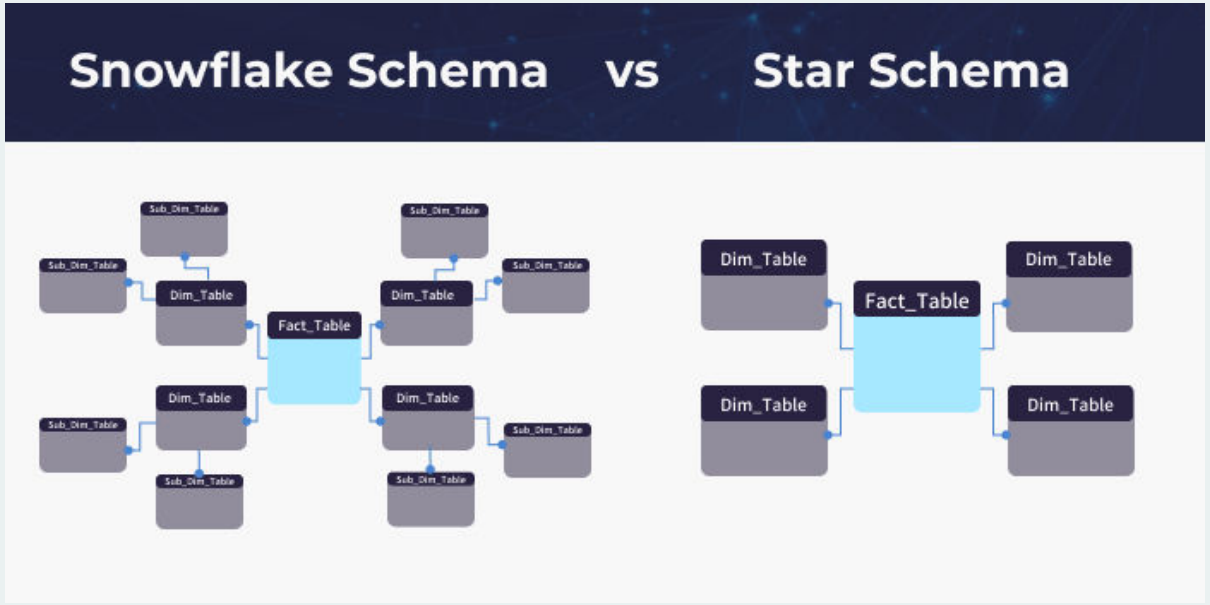
Differences between Cloud Data Lake and Cloud Data Warehouse

Feature	Cloud Data Lake	Cloud Data Warehouse
Data Type	Raw, unstructured data	Structured, processed data
Approach	Data-first approach	Problem-first approach
Data Storage	Highly scalable storage system	Requires predefined schema and data models
Processing	Schema-on-demand during processing	Optimized for fast querying and reporting
Suitable Data Sources	Diverse data sources and formats	Structured data from known sources
Ideal Use Cases	Big data, machine learning, advanced analytics	Business intelligence, reporting, and analytics

Data Warehouse Schemas

The structure of a Data Warehouse (in the Cloud or not) often follows one of those two structures:

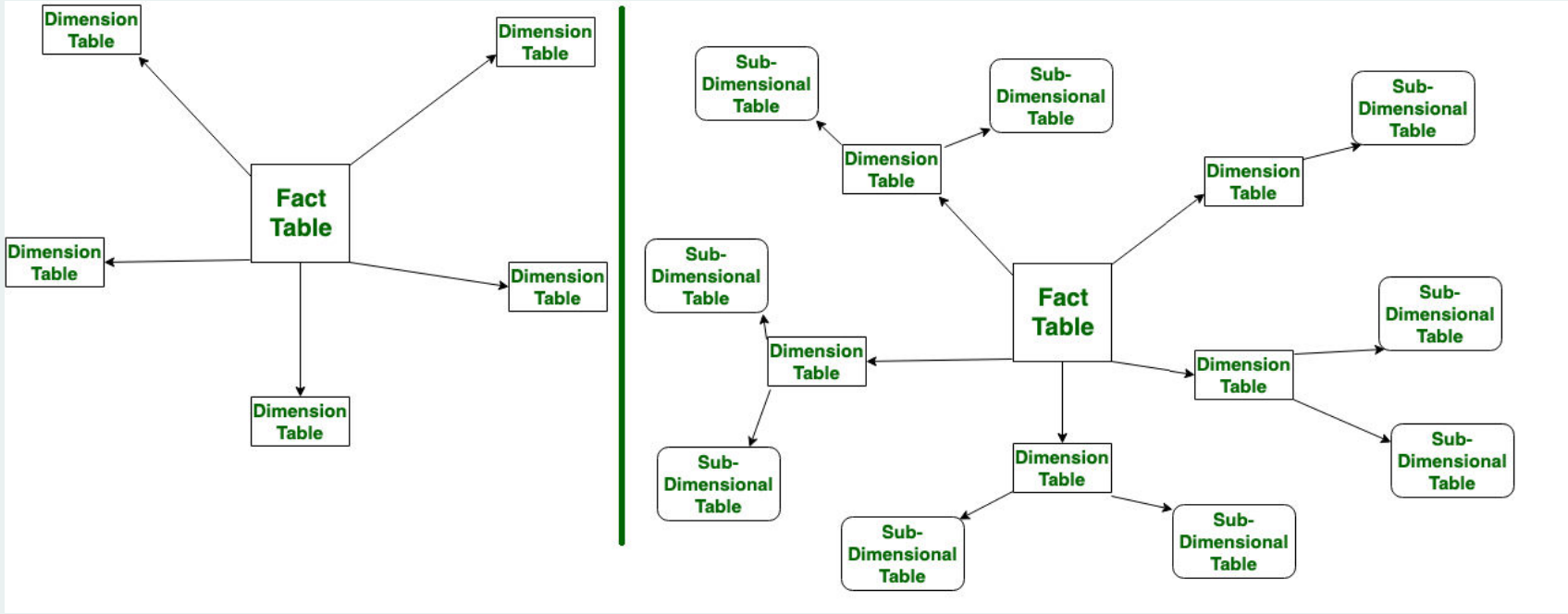
- **The Star Schema** ✨
- **The Snowflake Schema** ❄️



Star & Snowflake

Each of them is composed of:

- **Fact Table**:
  - **Central table** containing stores the main data points (**facts**) of the business process
  - Contains **foreign keys** that link to **dimension tables**
- **Dimension Tables**:
  - Store descriptive attributes (dimensions) related to the facts



Star Schema

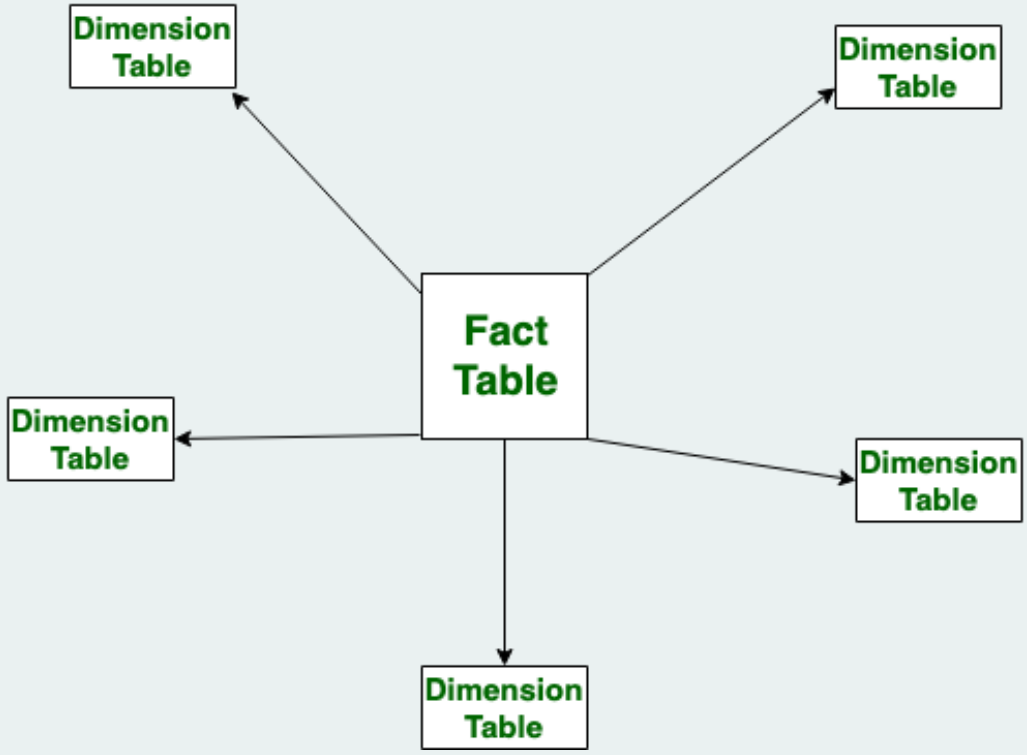
- **Each dimension table is connected to the fact table**

Advantages:

- Simplified queries and easier to understand
- De-normalized structure for faster query performance

Disadvantages:

- Data redundancy and increased storage requirements



Snowflake Schema

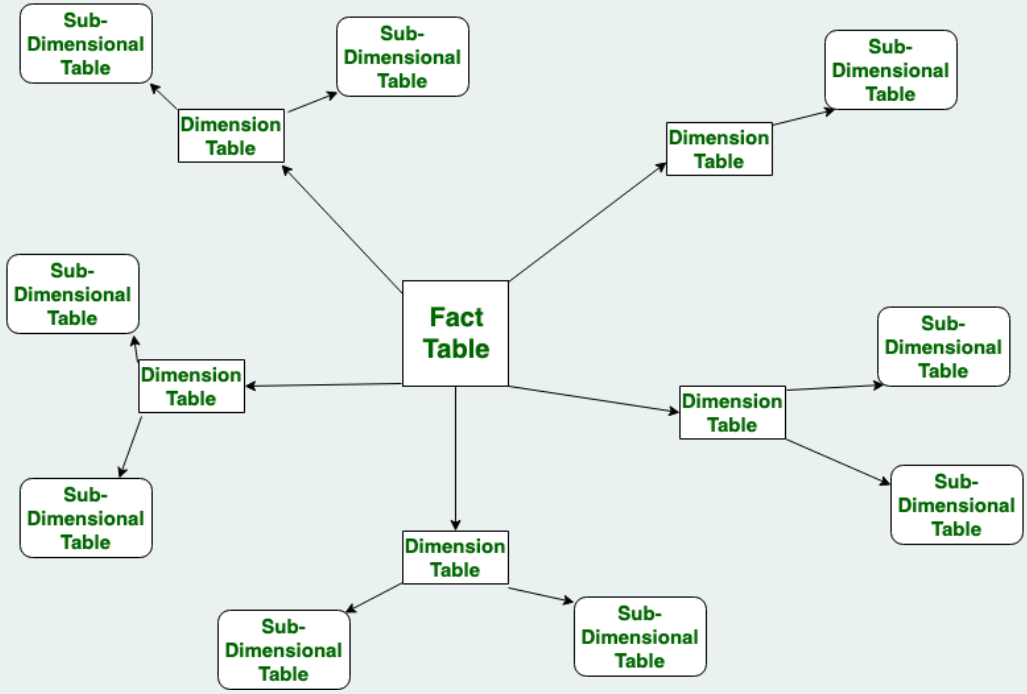
- **Each dimension table is connected to the fact table** and **normalized**, splitting data into **related sub-dimension tables**

Advantages:

- Reduced data redundancy and storage requirements
- Improved data integrity due to normalization

Disadvantages:

- More complex queries and joins
- Slower query performance due to additional joins



Existing technologies

Datalake Storage:







Datawarehouses:









Two patterns: ETL vs ELT

In case you did not see it already, the Data lake and Data Warehouse have two different philosophies on the way they handle the data:

ETL and ELT

ETL (Extract, Transform, Load)

- **Extract**: Data is extracted from various sources.
- **Transform**: Data is transformed into a format suitable for the data warehouse.
- **Load**: Transformed data is loaded into the data warehouse.

In this case, the `Transformation` happens before `Loading`: This requires more processing power at the transformation step because this means every single data is transformed.

However, the data is cleaner and easier to request: Better suited for a Data Warehouse

ELT (Extract, Load, Transform)

- **Extract**: Data is extracted from various sources.
- **Load**: Data is loaded into the data lake as-is.
- **Transform**: Data is transformed within the data lake when needed.

`Transformation` happens after `Loading`: This offers more flexibility for future data processing as data is stored as-is. However, this also means you have to spend time preparing the data before any utilization, which can be both time-consuming and computingly expensive.

Note on Costs: Storage vs compute

One important note: Nowadays it is often cheaper to store data rather than compute it.

This does not mean ELT is the way to go, you still have to take into account your needs and use case.

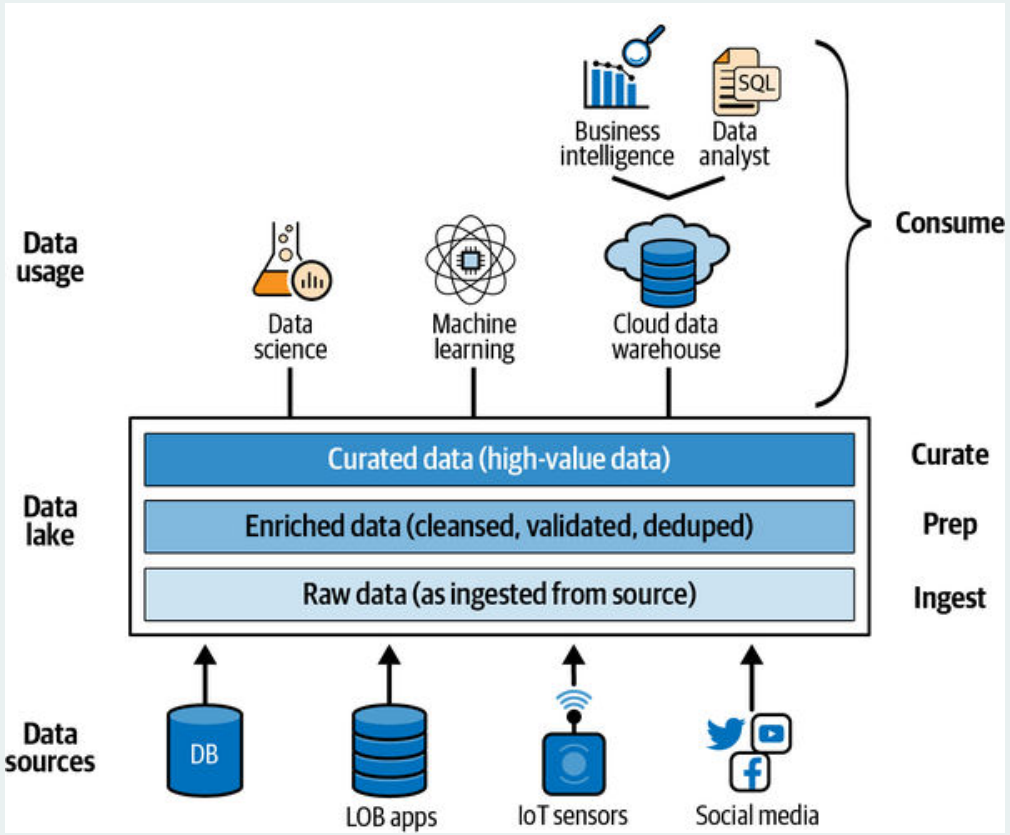


One Architecture to Rule Them All: The Modern Data Warehouse

In a **Modern Data Warehouse** architecture, both data lakes and data warehouses coexist, serving distinct purposes and complementing each other for a comprehensive data management solution.

This gives us the best of both worlds:

- **Data Lake**: Low-cost storage for large amounts of data; supports data science, machine learning, and exploratory scenarios
- **Data Warehouse**: Stores high-value data, powers business dashboards, and enables structured querying for insights



Benefits of The Modern Data Warehouse

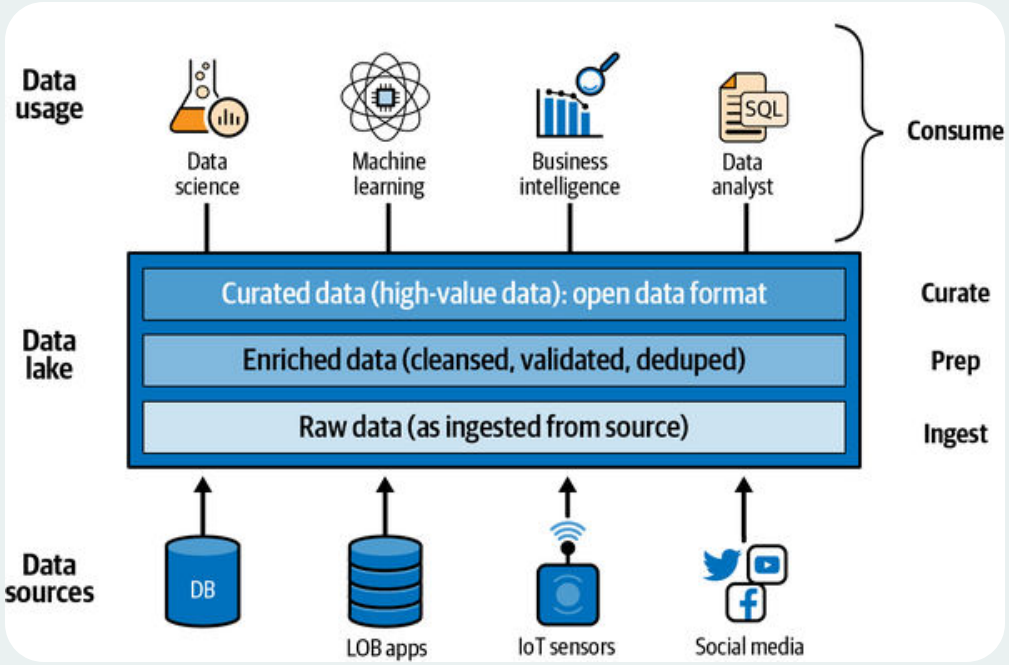
- **Optimized for SQL**: Leverage familiar SQL-based BI tool sets for business analysts.
- **ML & DS Scenarios**: Enable advanced data science and machine learning scenarios with data lakes.
- **Cost Reduction**: Lower costs by storing operational data in a data lake for longer periods (cold storage).
- **ACID Compliant**:These properties are essential for maintaining the integrity of high-value data

Challenges of The Modern Data Warehouse

- **Two Infrastructures**: Maintain both data lake and data warehouse infrastructures.
- **Data Management**: Ensure data quality and avoid turning the data lake into a data swamp.
- **Data Dependency**: Rely on data engineers to process and load new datasets into the warehouse.
- **Data Sharing Complexity**: Load data back into the data lake for exploratory analysis, increasing sharing complexity.

Lake House architecture

A Data Lake House is a hybrid architecture that combines the benefits of the data lakes and data warehouses like the Modern Data Warehouse, but in this case, the BI and Analysis are done directly on the data lake.



It aims to provide a unified platform for both structured and unstructured data.

Advanced file format

In order to combine both advantages, without the shortcomings of the datalake (unstructured data), engineers had to create new solutions: file formats that are compatible with the Data Lake storage, and with a structure that follows principles of Data Warehouse (ACID, defined schema ...)



They all have a **defined schema**, **ACID-ish compliance**, **metadata handling**, **schema enforcement**, **schema evolution** and are optimized to be requested in **SQL**.

We will get back to this in a bit.

Lake House Pros & Cons

Key Benefits:

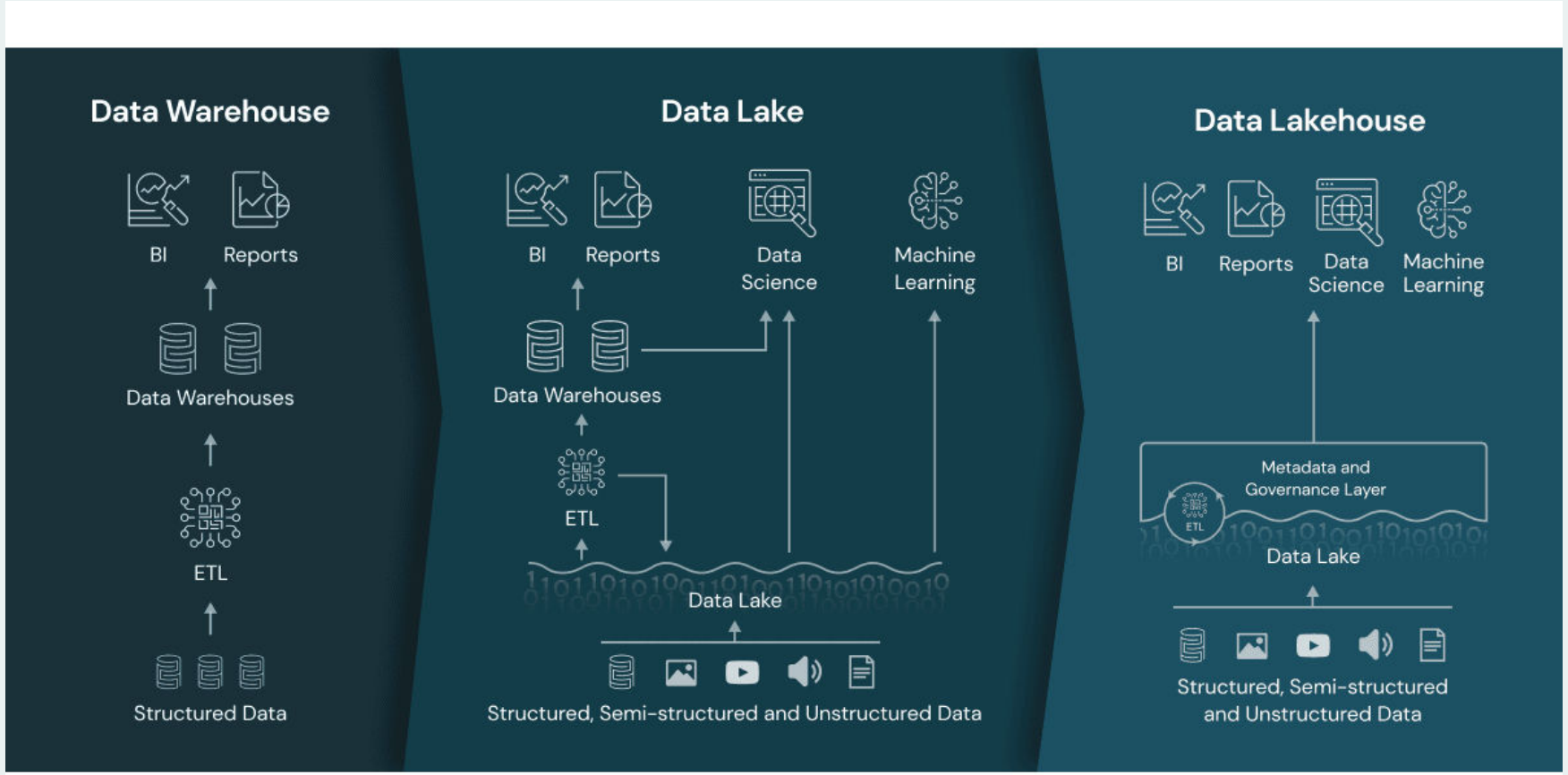
- Performant BI/SQL-based scenarios directly on the data lake
- Supports exploratory data science and machine learning scenarios
- Promotes sharing across various segments of data platform users
- Cost-effective compared to data warehouses

Challenges:

- Requires careful and difficult design and architecture for optimization
- Proper planning is essential to avoid issues
- Data warehouses offer optimized paths but are not truly open

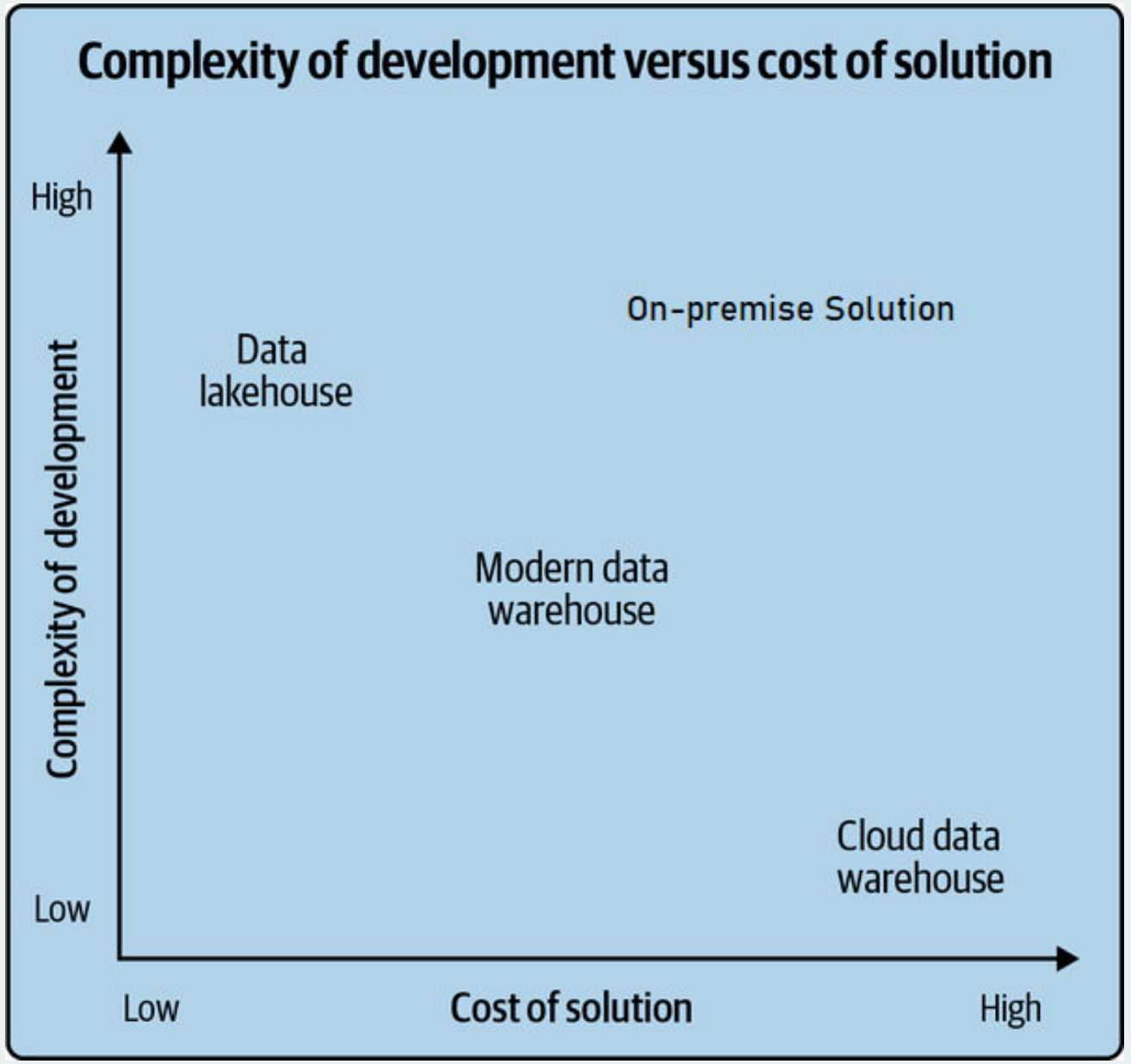
Summary of Cloud Data Platforms

Schema



Summary of Cloud Data Platforms

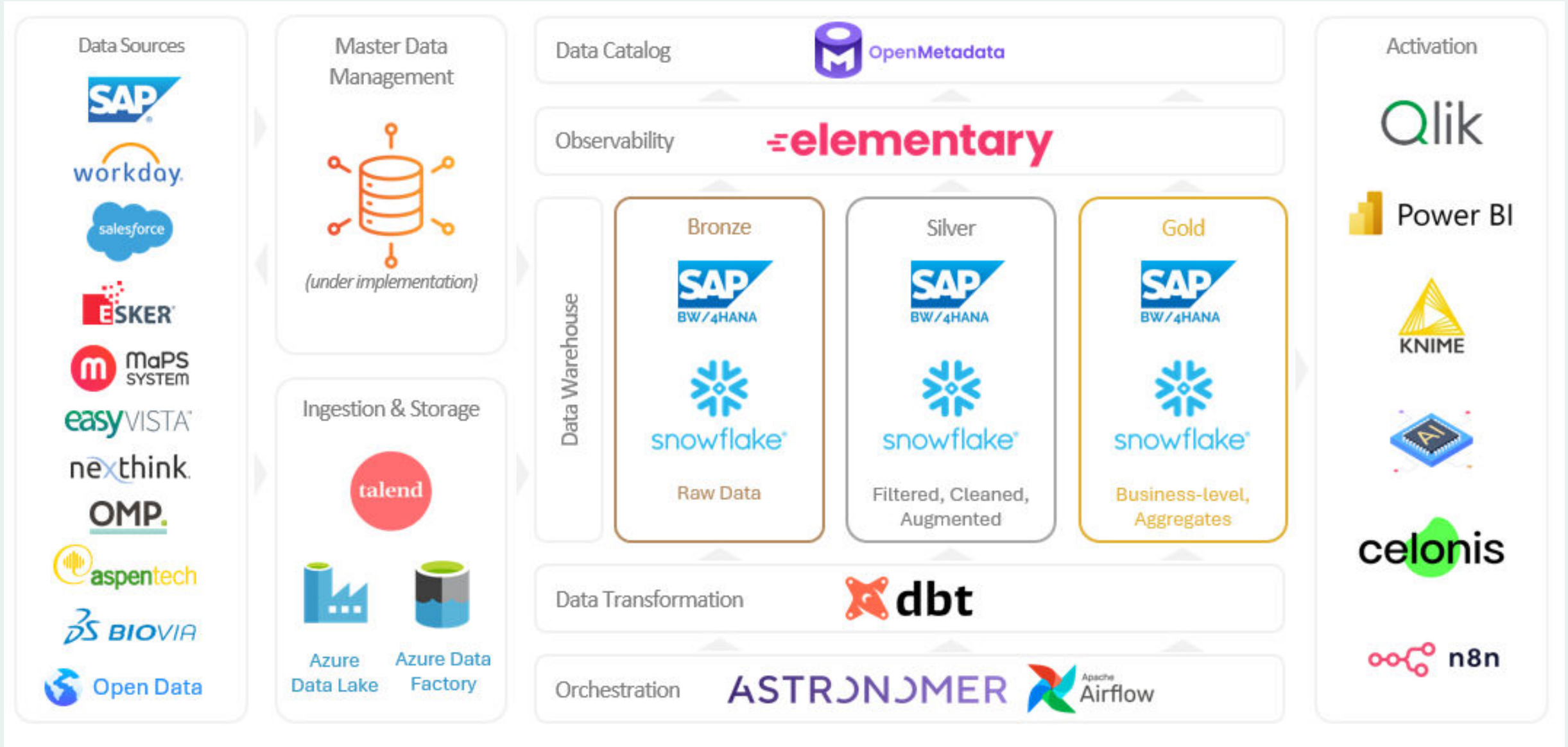
Cost & development costs



Comparison of maturity

Architecture	Total cost of solution	Flexibility of scenarios	Complexity of development	Maturity of ecosystem	Organizational maturity required
Cloud data warehouse	High	Low	Low	High for SQL/BI scenarios. Low for other scenarios.	Low
Modern data warehouse	Medium	Medium	Medium	Medium	Medium
Data lakehouse	Low	High	Medium to High	Medium to High	Medium to High

Example: Data Platform at Roquette



Advanced file format

Let's get back to the file format mentioned in the Lakehouse section.

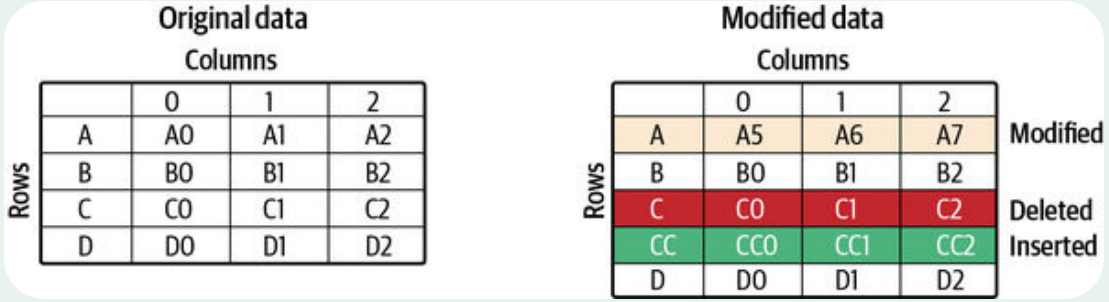
- 

- 

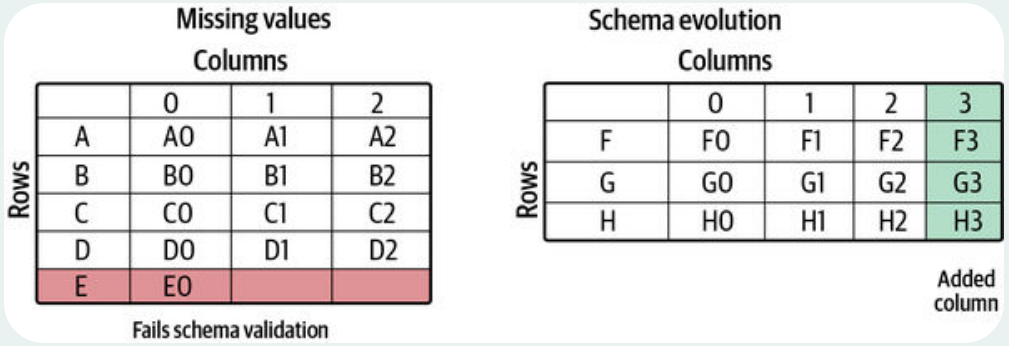
- 

Delta Lake overview

- Open-source storage layer based on Parquet
  - Added possibility of bulk update, delete and insert



  - Added possibility of schema validation and evolution

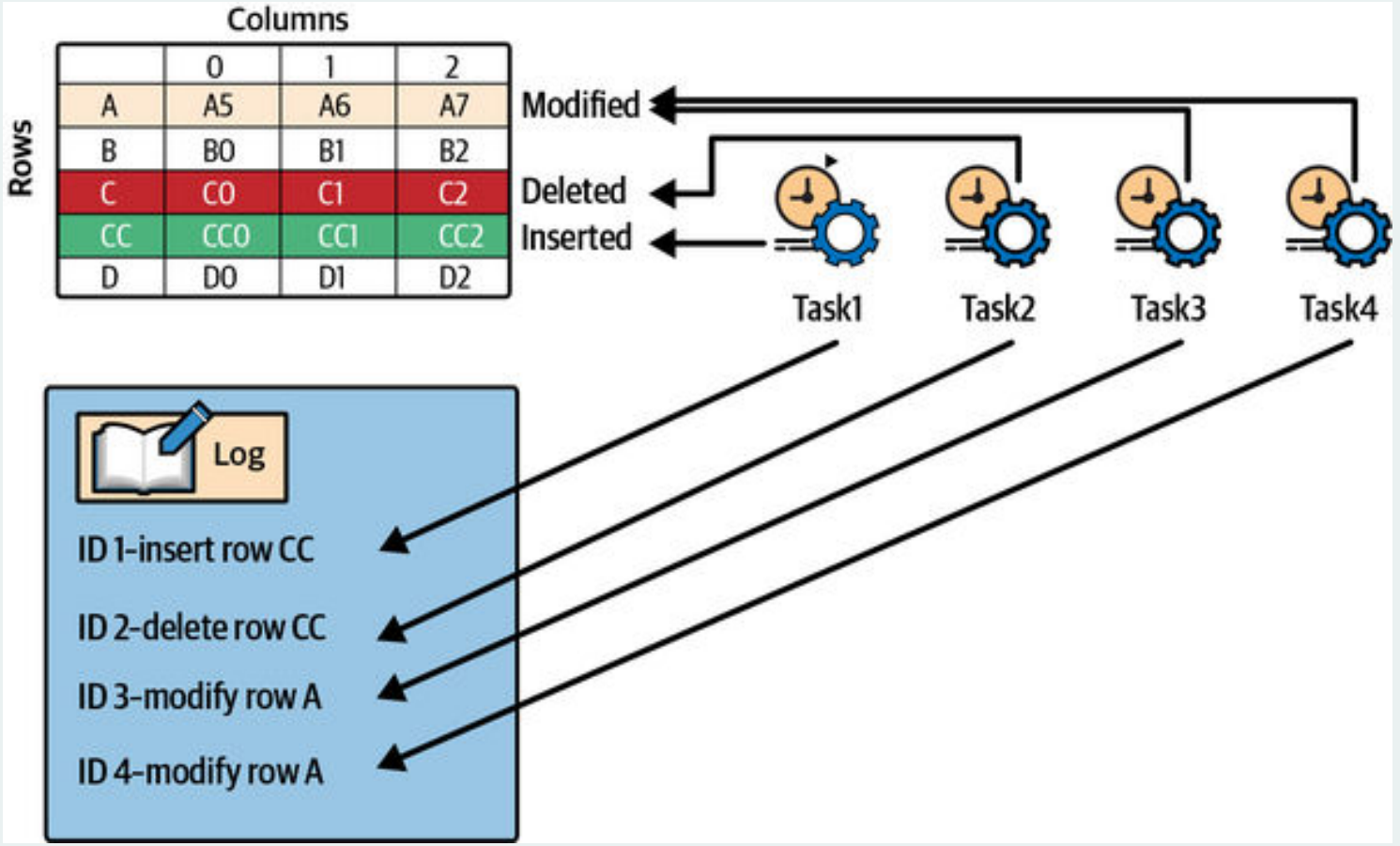


  - ACID capabilities
  - Time travel & versioning

How does Delta work?

Delta Lake is basically the combination of parquet and JSON files under the hood.

- The parquets files will store the data objects
- The JSON will store transaction logs to ensure the ACID capabilities (optimized for transactions).
- A JSON log checkpoint is created regularly to keep track of changes up to a certain time (optimized for storage)



Apache Iceberg

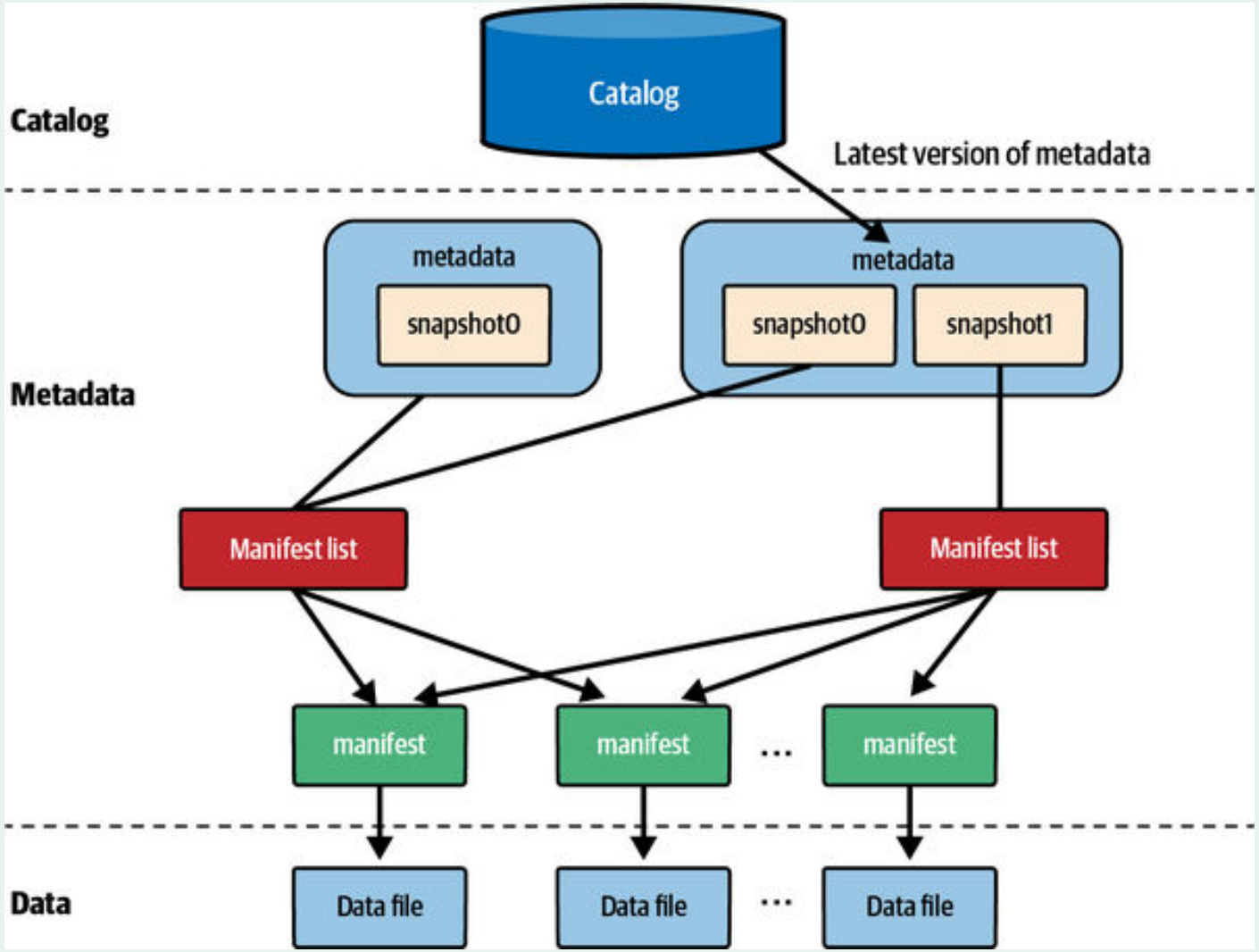
- Incubated by Netflix
- Open table format for huge analytic datasets
- Snapshot isolation, schema evolution, and partition evolution
- Designed for performance and scalability



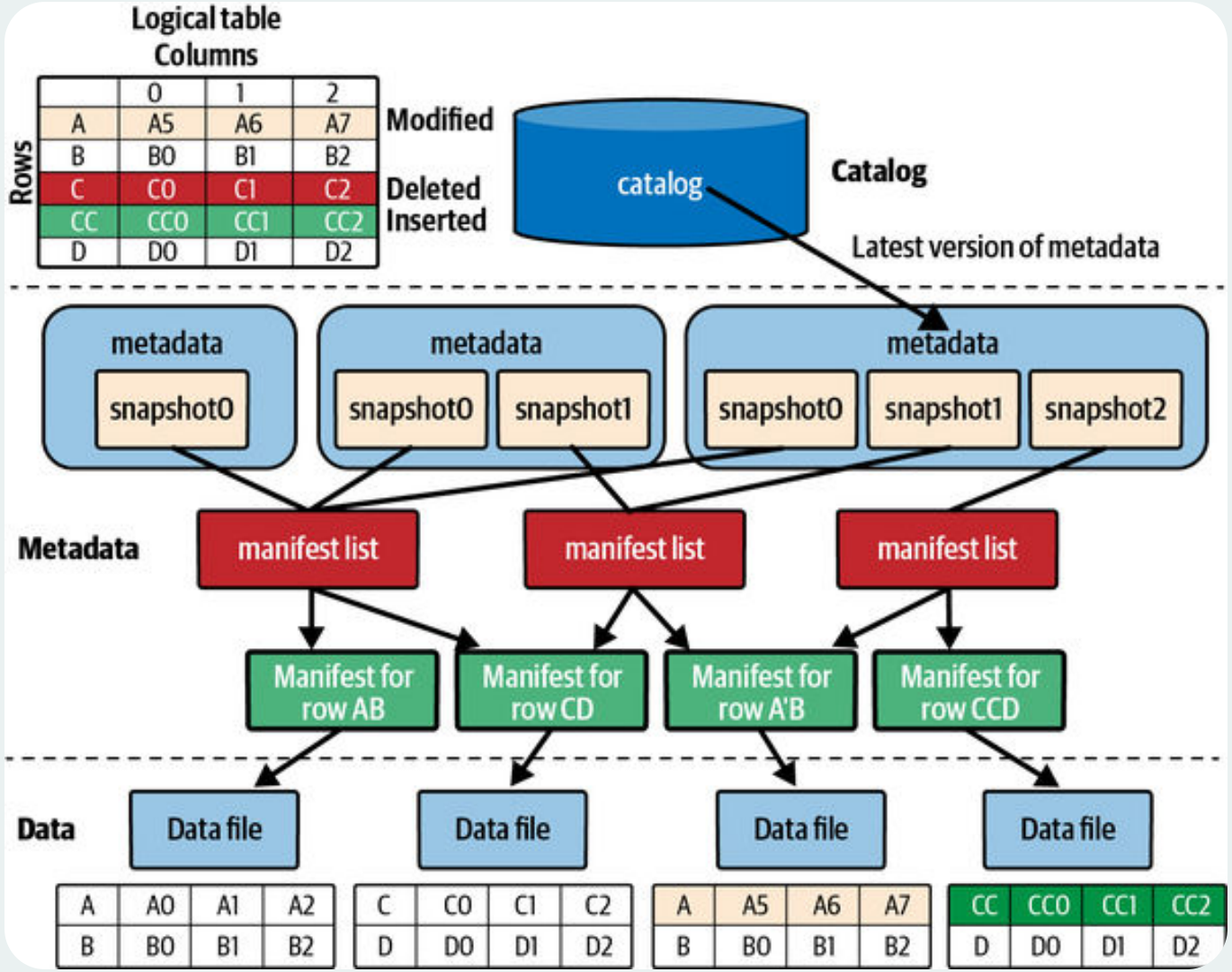
How does Iceberg work?

Like Delta, Iceberg is based on top of other file formats. An Iceberg file contains:

- The data files (Parquet and ORC)
- Metadata:
  - Snapshot metadata file that stores the metadata about the tables (schema, partitioning...)
  - Manifest files that contain paths to the files that actually store the data
  - Manifests lists
- A catalog file that has the pointers to the latest version of the metadata



A more in-depth look



Apache Hudi

- Incubated by Uber
- Open-source data management framework for data lakes
- Upsert/Delete support, incremental processing, and timeline management
- Built for large-scale, low-latency data processing

We will not go in depth with this one but is very similar to Iceberg and Delta in concepts.



Conclusion on file formats

- Each solution has its strengths and unique features
    - Delta Lake: Strong in versioning, time travel, and audit logging
    - Apache Iceberg: Excels in performance and scalability
    - Apache Hudi: Focused on low-latency data processing and timeline management

Credit and used resources

- Microsoft Azure documentation
- (Not Free) The Cloud Data Lake by Rukmani Gopalan
- DataBricks Medallion Architecture
- Row-Based vs Column-Based by Ashish Garg
- Oracle Understanding Slowly Changing Dimensions
- (French) Cyres Lambda & Kappa
- Hadoop & Spark courses from previous years by Olivier Moïse (2022) & Fabrice Woittequand (2020)